자율주행이동체 실제
(6. AUTOSAR)

충북대학교 지능로봇공학과
기석철 교수
sckee@cbnu.ac.kr, E8-7동 341-1호

AVLAB
AUTONOMOUS VEHICLE

Contents

1. **Autosar Overview**
2. **Autosar Architecture**
3. **Autosar Applications**

What is Autosar

AUTOSAR – AUTomotive Open Systems ARchitecture

- **Definition:** Middleware and system-level standard, jointly developed by automobile manufacturers, electronics and software suppliers and tool vendors.
- **Scale:** More than 100 members
- **Motto:** "*cooperate on standards, compete on implementations*"
- **Reality:** current struggle between OEM and Tier1 suppliers
- **Target:** facilitate portability, composability, integration of SW components over the lifetime of the vehicle

Standard of Components and Interface

The software implementing the automotive functionality is encapsulated in software components. Standardization of the interfaces is central to support scalability and transferability of functions. Any standard-conformant implementation of a software component can be integrated with substantially reduced effort in a system.
The standardization could be developed incrementally towards:

- **Level of abstraction**
  - Functional aspects
  - Behavior and implementation aspects
- **Level of decomposition**
  - Low degree of decomposition of the functional domain
  - High degree of decomposition of the functional domain
- **Level of architecture definition**
  - Terminology
  - Standardized data-types
  - Partial description of interfaces (without semantics)
  - Complete description of interfaces (without semantics)
  - Complete description of interfaces (with semantics)
  - Partial definition of the functional domain
  - Complete definition of the functional domain

Functional Domains

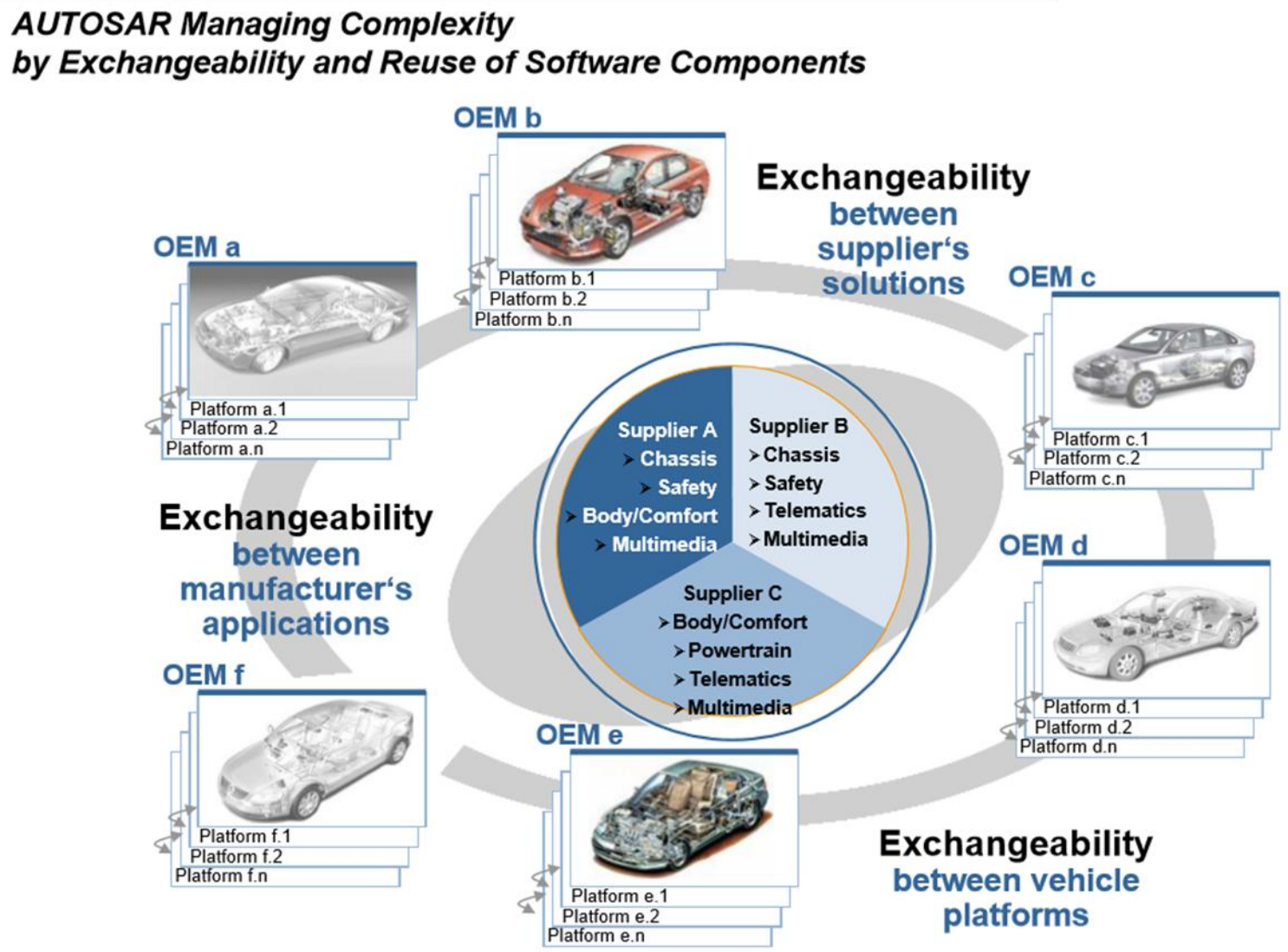
The specification of functional interfaces is divided into 6 domains:

- Body/Comfort
- Powertrain
- Chassis
- Safety
- Multimedia/Telematics
- Man-machine-interface

The domains could be differently handled due to intellectual property rights issues and decomposition levels.

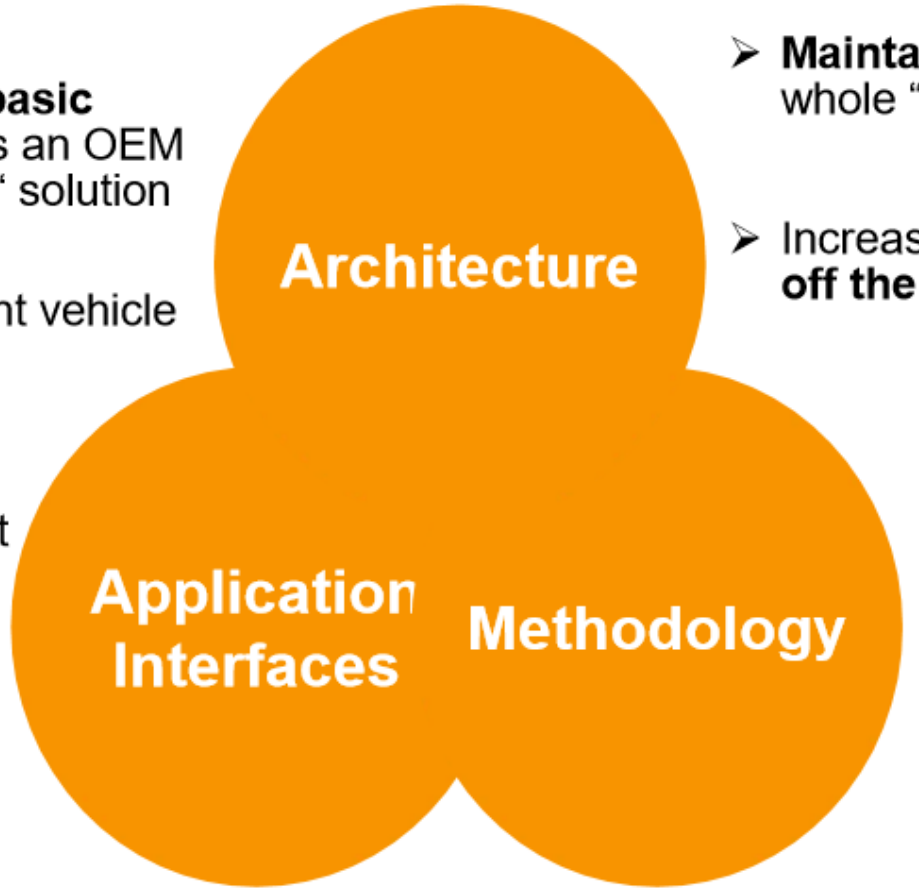
In the first phase of AUTOSAR only in the domains **body/comfort, chassis, and powertrain** results can be expected. All others have lower priority in the first phase.

Industrial Background



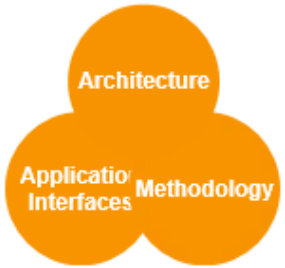
Main Topic

- Implementation and **standardization of basic system functions** as an OEM wide “Standard Core“ solution
- **Scalability** to different vehicle and platform variants
- **Transferability of functions** throughout network
- **Integration** of functional modules from **multiple suppliers**

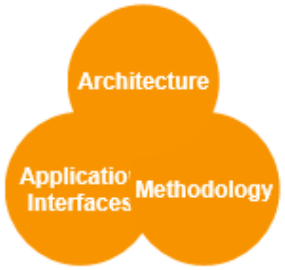


- **Maintainability** throughout the whole “Product Life Cycle“
- Increased use of **“Commercial off the shelf hardware“**
- **Software updates** and upgrades over vehicle lifetime
- Consideration of availability and **safety** requirements
- **Redundancy** activation

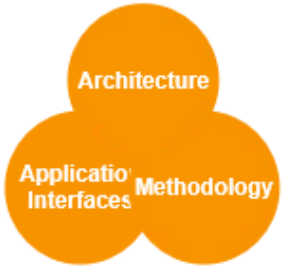
Main Topic



- **Architecture:**
  Software architecture including a complete basic or environmental software stack for ECUs – the so called AUTOSAR Basic Software – as an integration platform for hardware independent software applications.



- **Methodology:**
  Exchange formats or description templates to enable a seamless configuration process of the basic software stack and the integration of application software in ECUs and it includes even the methodology how to use this framework.



- **Application Interfaces:**
  Specification of interfaces of typical automotive applications from all domains in terms of syntax and semantics, which should serve as a standard for application software.

Benefits for industry

OEM

- OEM overlapping reuse of software modules
- Maintaining ability to compete on innovative functions, enlarged design flexibility
- Simplification of the integration task
- Reduction of total SW development costs

Supplier

- Reduction of version proliferation
- Development partitioning among suppliers
- Increase of efficiency in functional development
- New business models possible

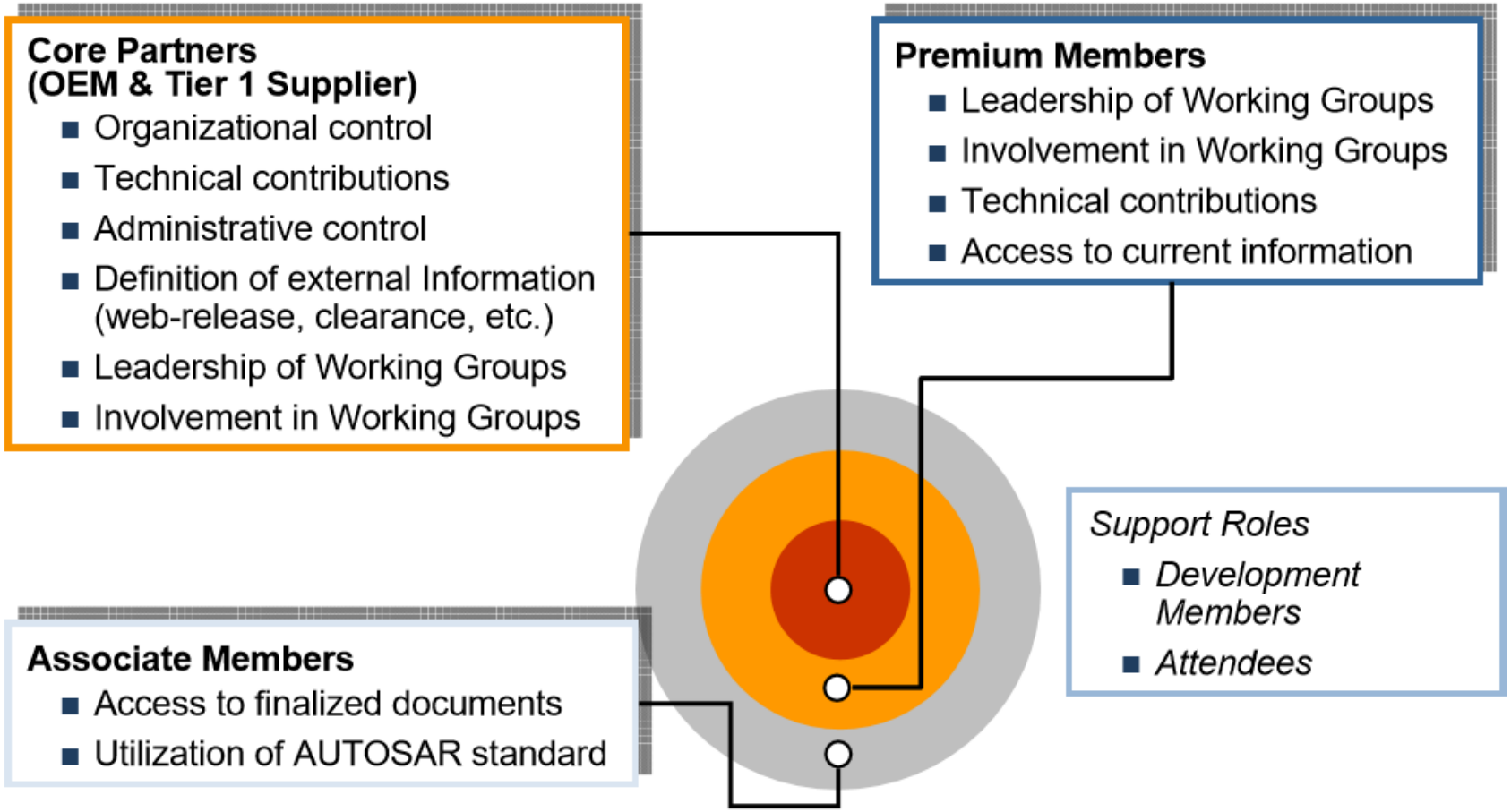
Tool provider

- Common interfaces with development processes
- Seamless, manageable, task optimized (time dependent) tool landscape

New market entrant

- Transparent and defined interfaces enable new business models

Partnership Structure

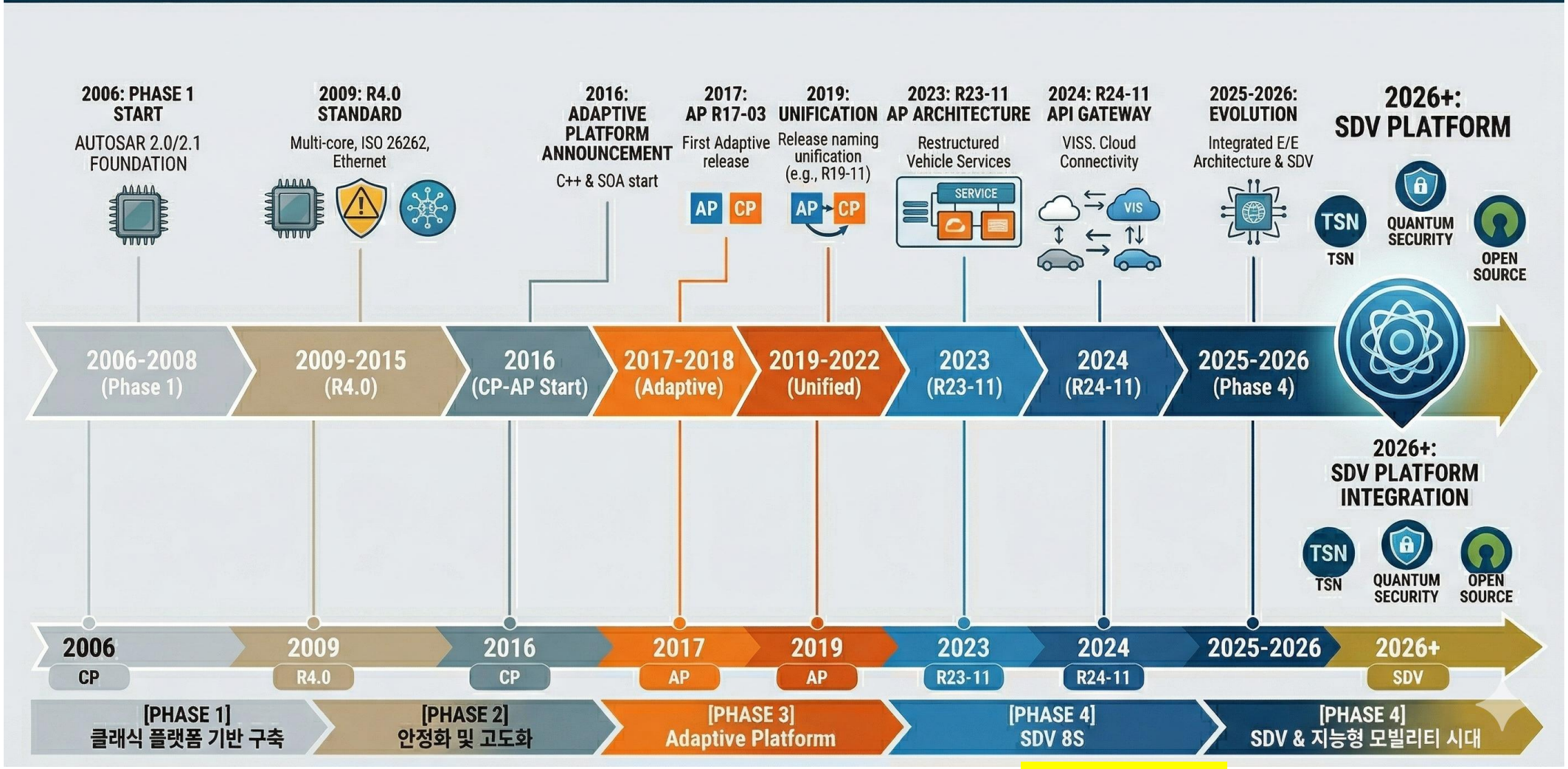


Core partners and members

Up-to-date status see: http://www.autosar.org, 2024년 4월까지 350+ 기관 가입

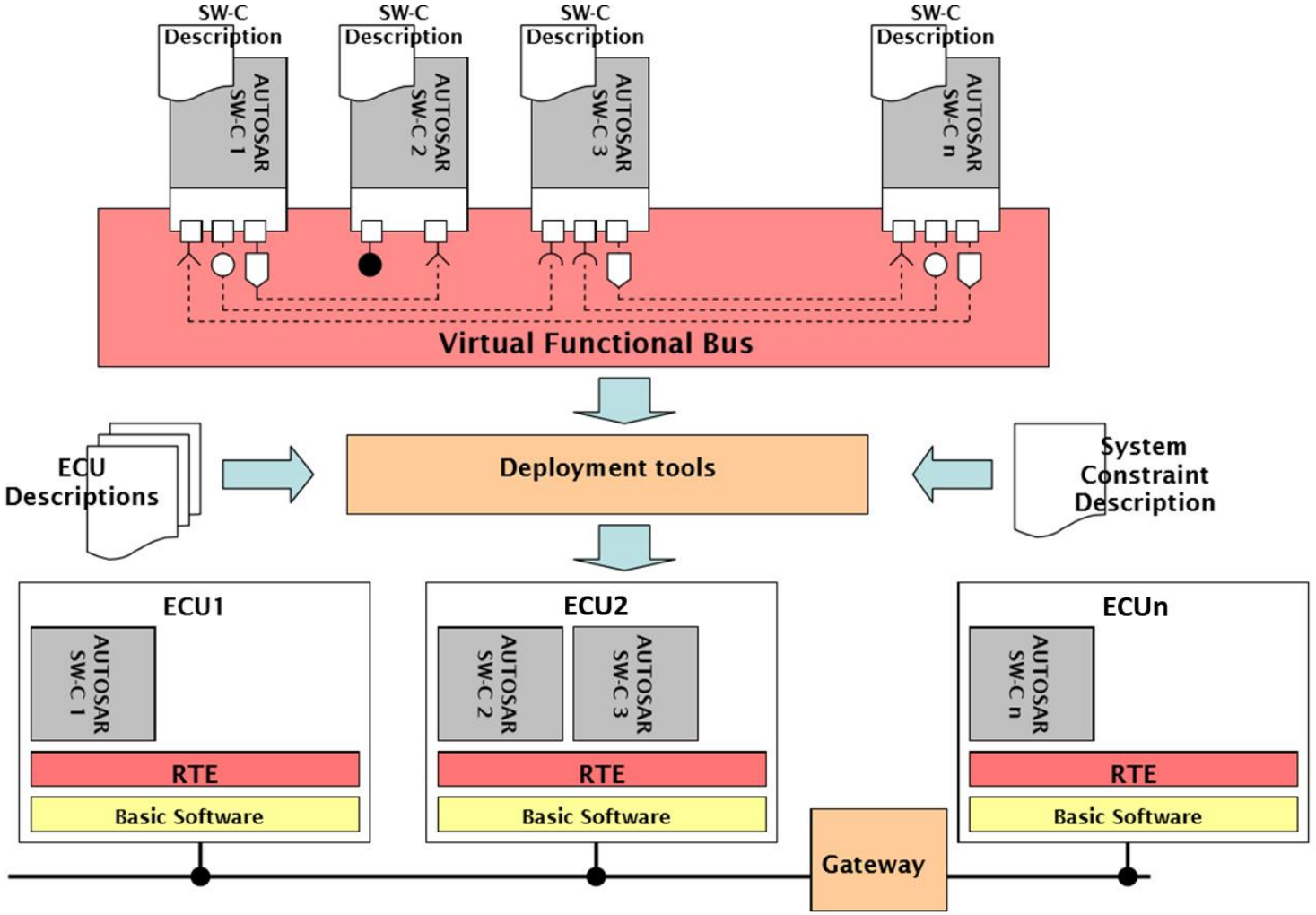
AVLAB AUTONOMOUS VEHICLE

AUTOSAR Roadmap



8-Layer SDV Stack

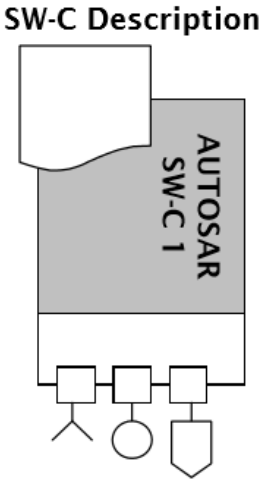
Architecture



Software Component

The AUTOSAR Software Components encapsulate an application which runs on the AUTOSAR infrastructure. The AUTOSAR SW-C have well-defined interfaces, which are described and standardized.

- **SW-C Description**
  - For the interfaces as well as other aspects needed for the integration of the AUTOSAR Software Components, AUTOSAR provides a standard description format (SW-C Description).

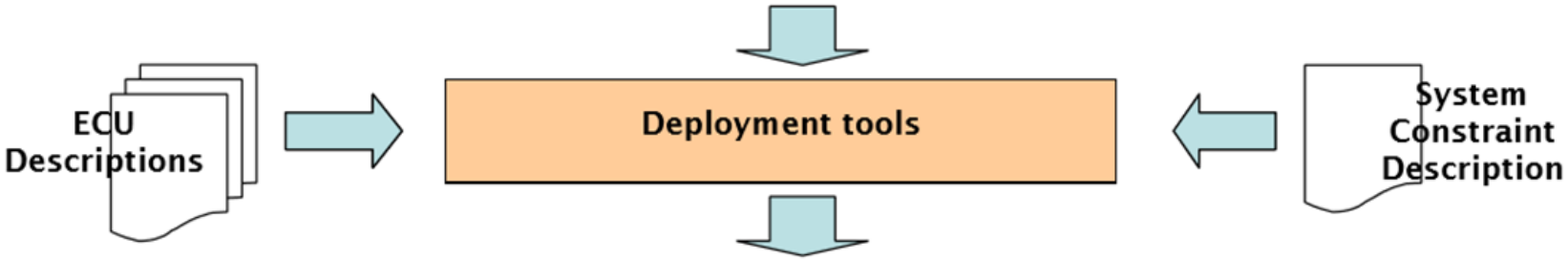


Virtual Functional Bus(VFB)

- The “virtual functional bus” is the communication mechanism that allows these components to interact.
- In a design step called “Configure System”, the components are mapped on specific system resources (ECUs). Thereby, the virtual connections between the components are mapped onto local connections (within a single ECU) or on network-technology specific communication mechanisms (such as CAN or FlexRay frames).
- Finally, the individual ECUs in such a system can be configured.
- The concrete interface between the individual components and between the components and the Basic Software (BSW) is called the Run-Time Environment (RTE).
- The VFB specification needs to provide concepts for all infrastructure-services that are needed by a component implementing an automotive application. These include:
  - Communication to other components in the system
  - Communication to sensors and actuators in the system
  - Access to standardized services, such as reading to or writing from non-volatile ram
  - Responding to mode-changes, such as changes in the power-status of the local ECU
  - Interacting with calibration and measurement systems

ECU Descriptions

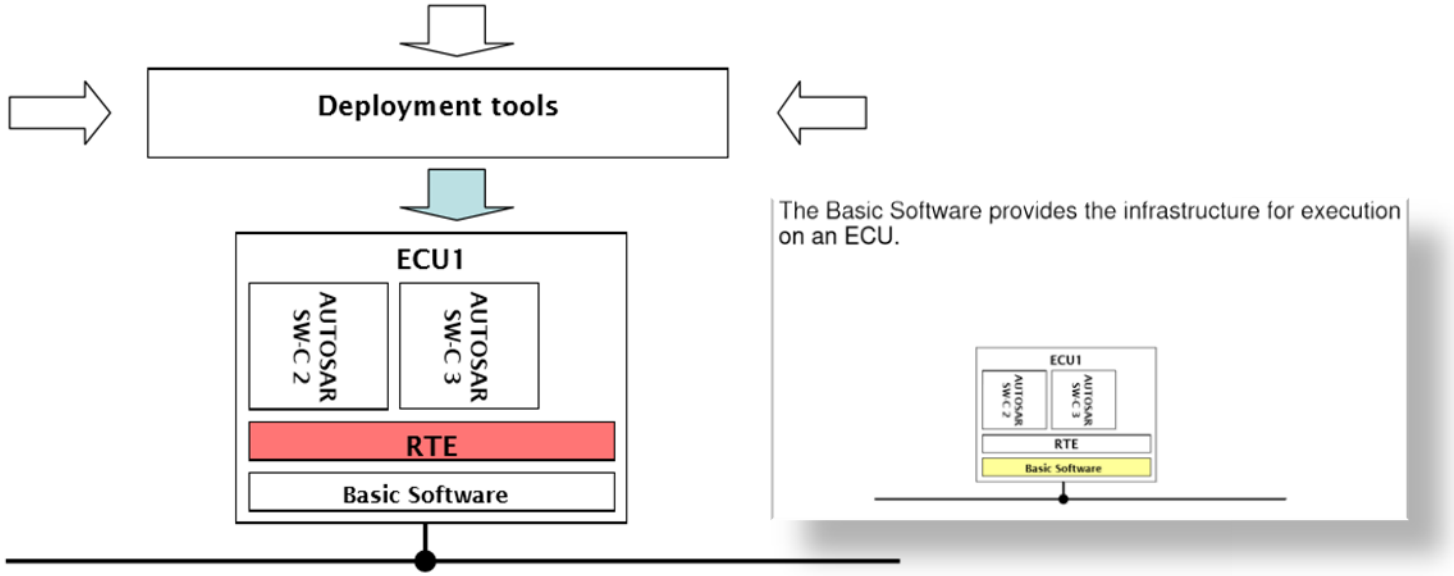
In order to integrate AUTOSAR SW-Components into a network of ECUs, AUTOSAR provides description formats for the system as well as for the resources and the configuration of the ECUs.



Mapping on EUCs

AUTOSAR defines the methodology and tool support to build a concrete system of ECUs. This includes the configuration and generation of the Runtime Environment (RTE) and the Basic Software (RTOS) on each ECU.

- **Runtime Environment (RTE)**
  - From the viewpoint of the AUTOSAR Software Component, the RTE implements the VFB functionality on a specific ECU.



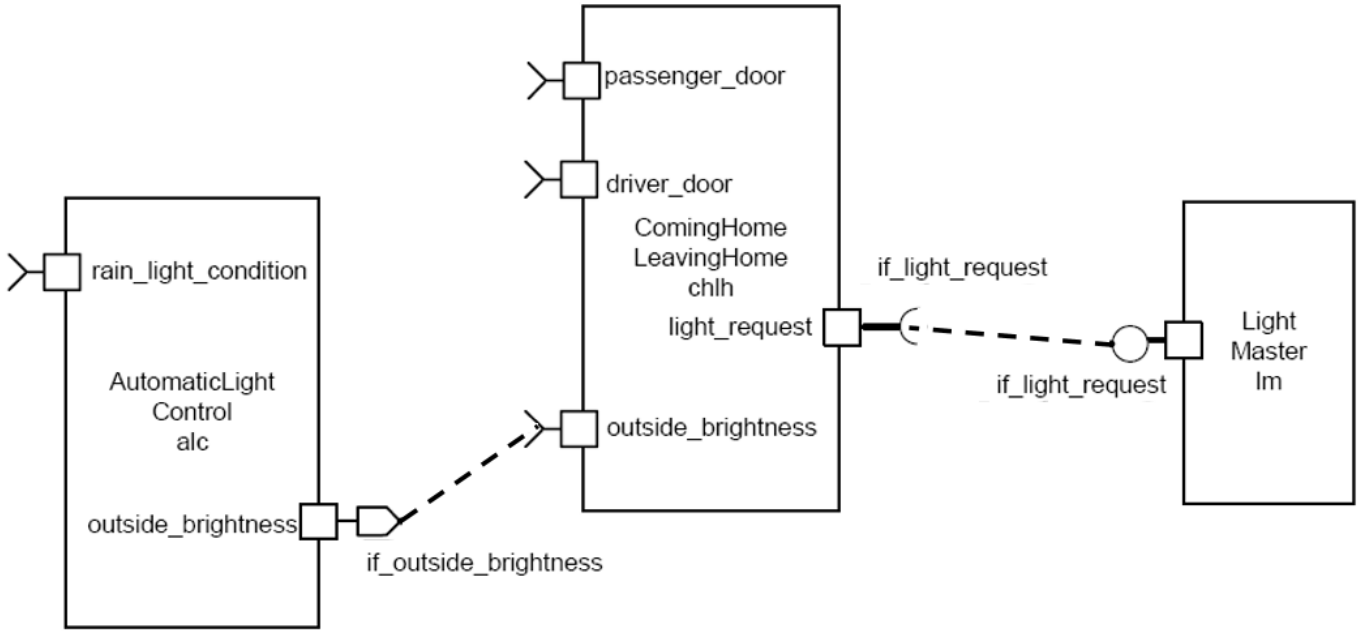
Architecture

Core Concept of AUTOSAR

A fundamental concept of AUTOSAR is the separation between:

- **application** and
- **infrastructure**.

An application in AUTOSAR consists of Software Components interconnected by connectors.



Architecture

The generic “AUTOSAR Component” concept

- **AUTOSAR Software Component**
- **Sensor/Actuator Software Component (special case)**
- **Composition**
  - A logical interconnection of components packaged as a component. In contrast to the Atomic Software Components, the components inside a composition can be distributed over several ECUs.
- **ECU Abstraction**
- **Complex Device Driver**
- **AUTOSAR Services**

Software Component

Each AUTOSAR Software Component encapsulates part of the functionality of the application.

- AUTOSAR does not prescribe the granularity of Software Components. Depending on the requirements of the application domain an AUTOSAR Software Component **might be a small**, reusable piece of functionality (such as a filter) or **a larger block** encapsulating an entire subsystem.

The AUTOSAR Software Component is an "Atomic Software Component"

- *Atomic* means that the each instance of an AUTOSAR Software Component is **statically assigned to one ECU.**

Implementing an AUTOSAR Software Component

AUTOSAR does not prescribe **HOW** an AUTOSAR Software Component should be implemented

- a component may be handwritten or generated from a model

Software Component

Shipping an AUTOSAR Software Component

A shipment of an AUTOSAR Software Component consists of:

- A complete and formal **Software Component Description** which specifies how the infrastructure must be configured for the component, and

- An **implementation** of the component, which could be provided as "object code" or "source code".

SWC - Description

The AUTOSAR Software Component Description contains:

- **the operations and data elements** that the software component provides and requires
  - described using the **PortInterface** concept
- **the requirements** on the infrastructure,
- **the resources** needed by the software component (memory, CPU-time, etc.),
- **information** regarding the specific implementation of the software component.
- The structure and format of this description is called **“software component template”**.

SWC - Implementation

A source code component implementation is independent from

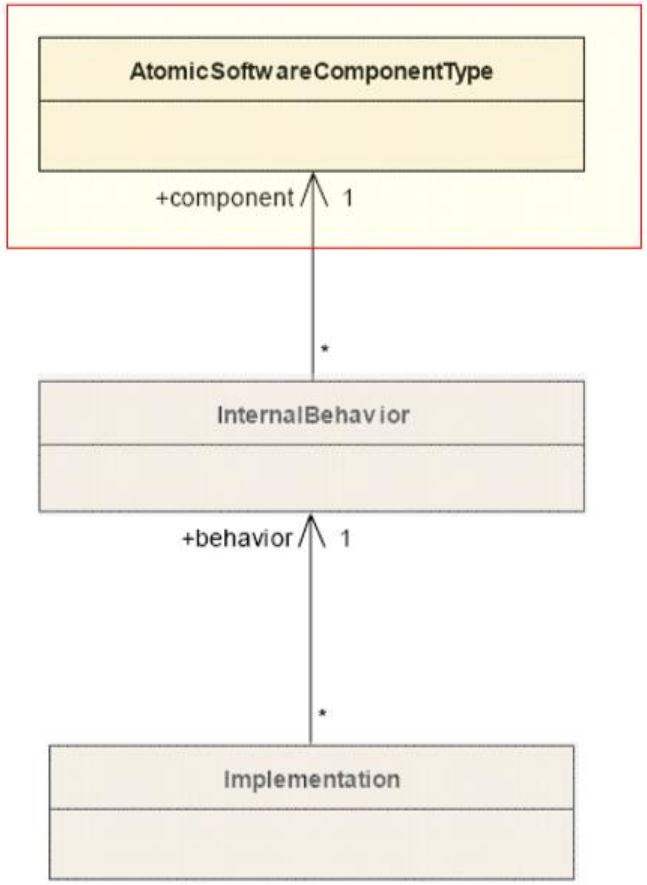
- **the type of microcontroller of the ECU and the type of ECU on which the component is mapped**
  - The AUTOSAR infrastructure takes care of providing the software component with a standardized view on the ECU hardware
- **the location of the other components with which it interacts.**
  - The component description defines the data or services that it provides or requires. The component doesn't know if they are provided from components on the same ECU or from components on a different ECU.
- **the number of times a software component is instantiated in a system or within one ECU**

SWC – Description level

The highest (most abstract) description level is the **Virtual Functional Bus**.

Here components are described with the means of datatypes and interfaces, ports and connections between them, as well as hierarchical components. At this level, the fundamental communication properties of components and their communication relationships among each other are expressed.

- **Software components**
- **Compositions**
- **Interfaces**

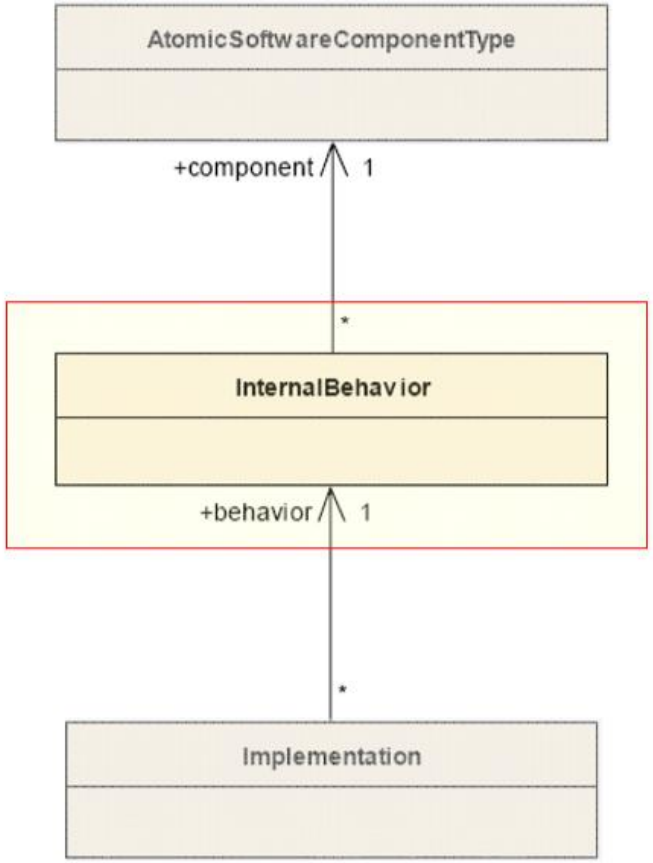


SWC – Description level

Description of components on RTE level

The middle level allows for behavior description of a given component. This behavior is expressed through RTE concepts, e.g. **RTE events** and in terms of **schedulable units(Runnables).** For instance, for an operation defined in an interface on the VFB, the behavior specifies which of those units is activated as a consequence of the invocation of that operation.

- **Runnables**
- **Events**
- **Interaction with the Run Time Environment**



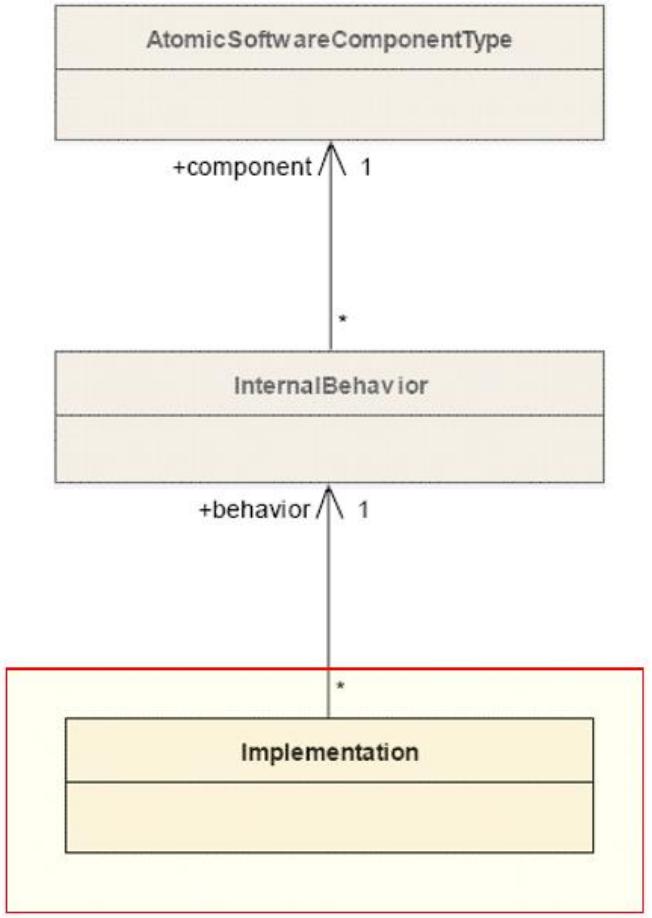
SWC – Description level

Descriptions of components on implementation level

The lowest (most concrete) level of description specifies the implementation of a given behavior. More precisely, the **schedulable units** of such a behavior are **mapped to code**.

The two layers above constrain the RTE API that a component is offered, the implementation now utilizes this API.

- **Component implementation**
- **Resource consumption of SW-Components**



Components, Ports, Interfaces

- A component has well-defined **ports**, through which the component can interact with other components.
- A port always belongs to exactly one component and represents a point of interaction between a component and other components.
- To define the services or data that are provided on or required by a port of a component, the **AUTOSAR Interface** concept is introduced.
- The AUTOSAR Interface can be:
  - **Client-Server** Interface defining a set of operations that can be invoked
  - **Sender-Receiver** Interface, for data-oriented communication

Components, Ports, Interfaces

- A port can be
  - **PPort** (provided interface)
  - **RPort** (required interface)
- **When a PPort *provides* an interface, the component to which the port belongs**
  - provides an implementation of the operations defined in the **Client-Server Interface**
  - generates the data described in a data-oriented **Sender-Receiver Interface**.
- **When an RPort of a component *requires* an AUTOSAR Interface, the component can**
  - invoke the operations when the interface is a **Client-Server**
  - read the data elements described in the **Sender-Receiver Interface**.

Communication Patterns

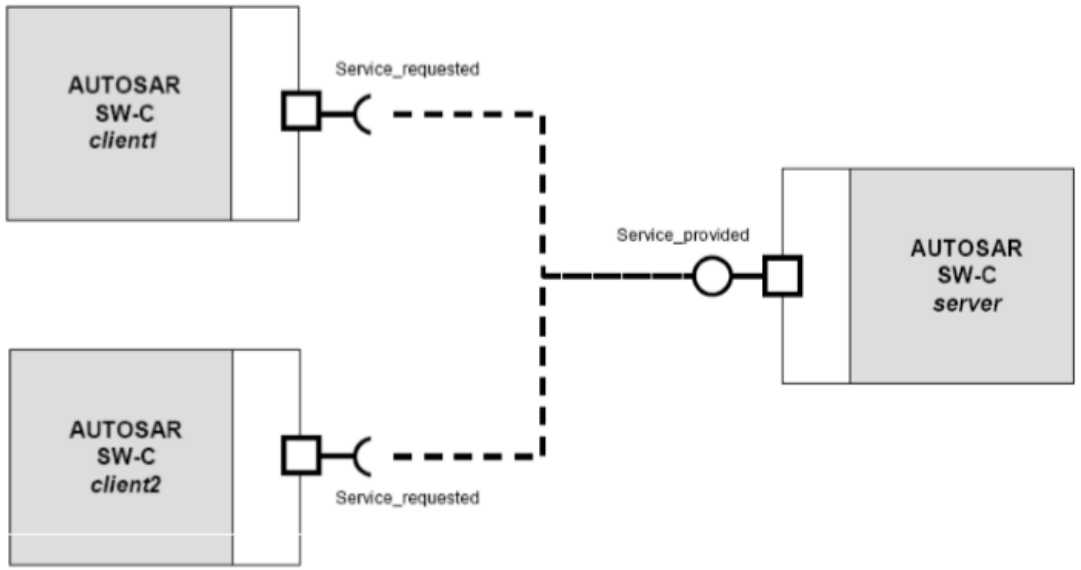
- **Elementary Communication Patterns**
  - **Client-Server**
  - **Sender-Receiver**
- **Interfaces specify**
  - which services with which arguments can be called by client-server communication
  - what information sender receiver communication transports
- The formal description of the interface is in the **software component template**, including also data types that can be used and interface compatibility.
- The **detailed behavior of a basic communication pattern is specified by attributes**. With those attributes e.g. the length of data queues and the behavior of receivers (blocking, non-blocking, etc.) and senders (send cyclic, etc.) can be defined.

Client - Server

- The server is a provider and the client is a user of a service.
- The client initiates the communication, requesting that the server performs a service, transferring a parameter set if necessary. The server waits for incoming communication requests from a client, performs the requested service and dispatches a response to the client's request.
- The direction of initiation is used to categorize whether an AUTOSAR Software Component is a client or a server. A single component can be both a client and a server depending on the software realization.
- After the service request is initiated and until the response of the server is received The client can be:
  - **blocked** (synchronous communication)
  - **non-blocked** (asynchronous communication).

Client - Server

An example of client-server communication in the composition of three software components and two connections in the VFB model view.

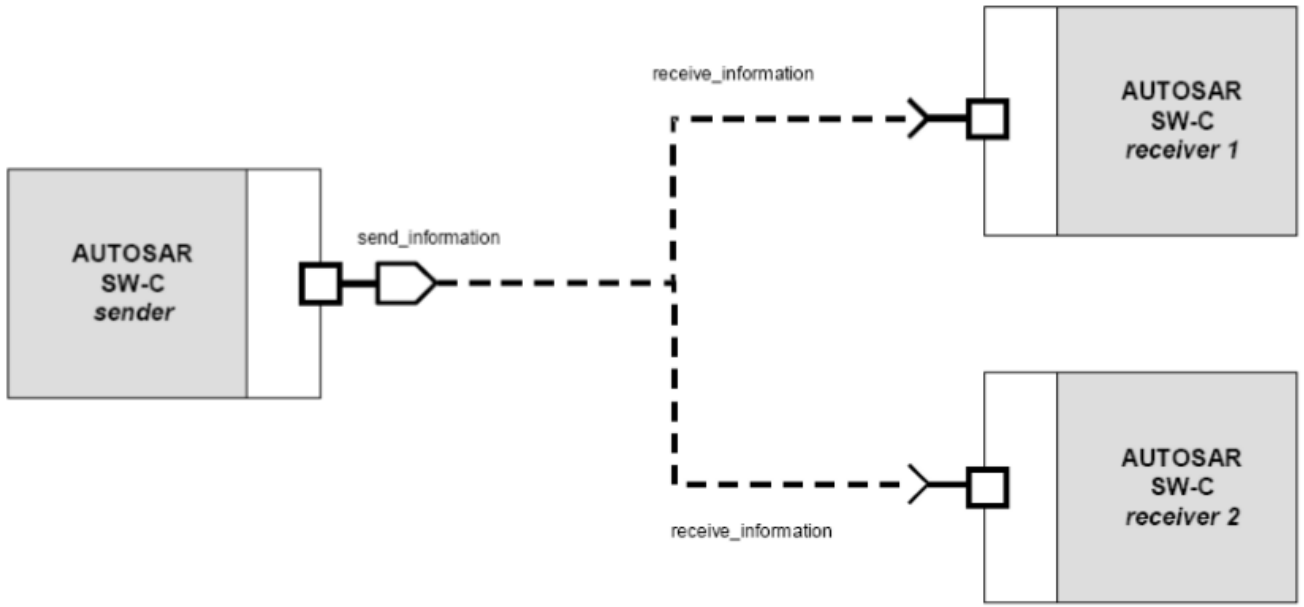


Sender - Receiver

- Model for the asynchronous distribution of information where a sender distributes information to one or several receivers.
- The sender is not blocked (asynchronous communication) and neither expects nor gets a response from the receivers (data or control flow), the sender just provides the information and the receivers decides autonomously when and how to use it.
- It is the responsibility of the communication infrastructure to distribute the information.
- The sender does not know the identity or the number of receivers.

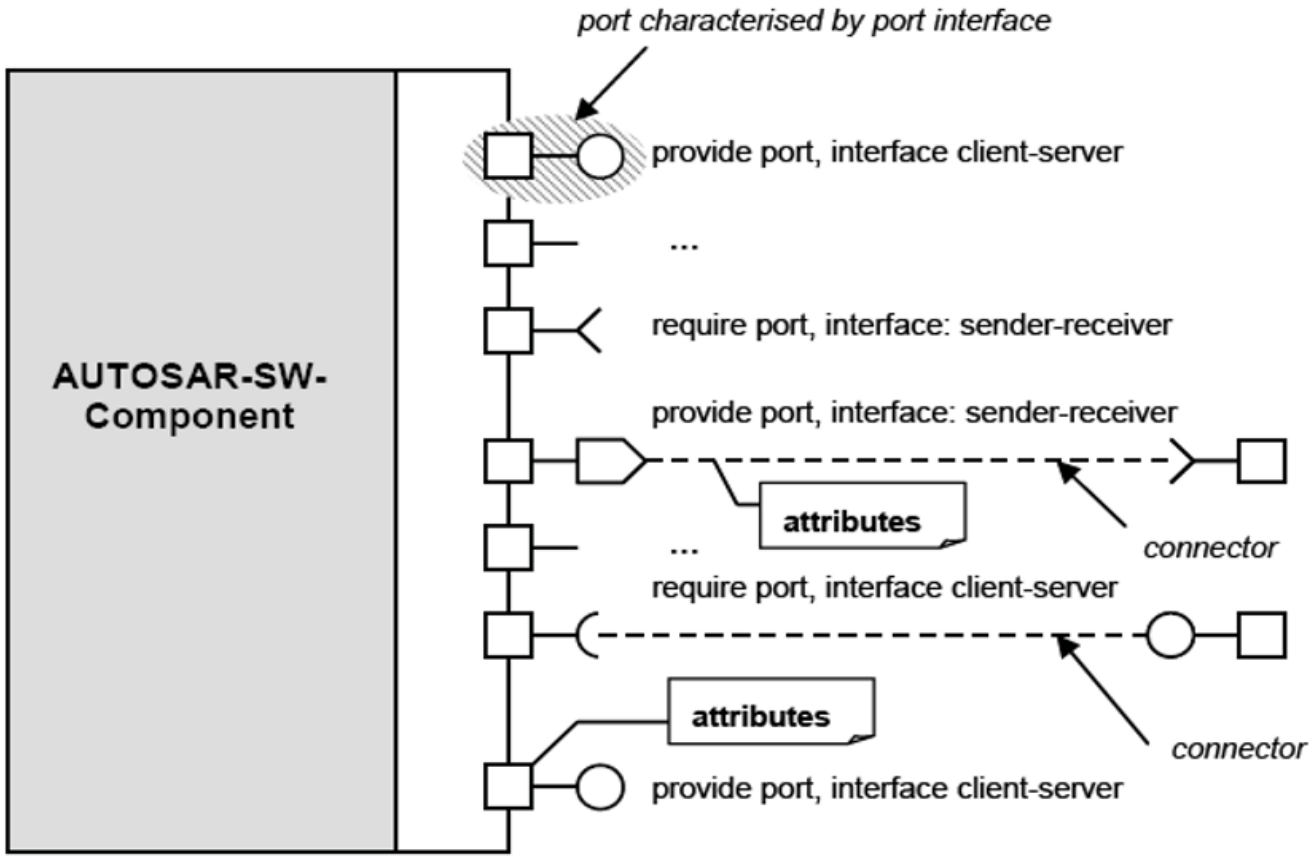
Sender - Receiver

an example of how sender-receiver communication is modeled in AUTOSAR



Component Model

The visual representation of components, ports and interfaces in a component model



Communication via the Virtual Functional Bus

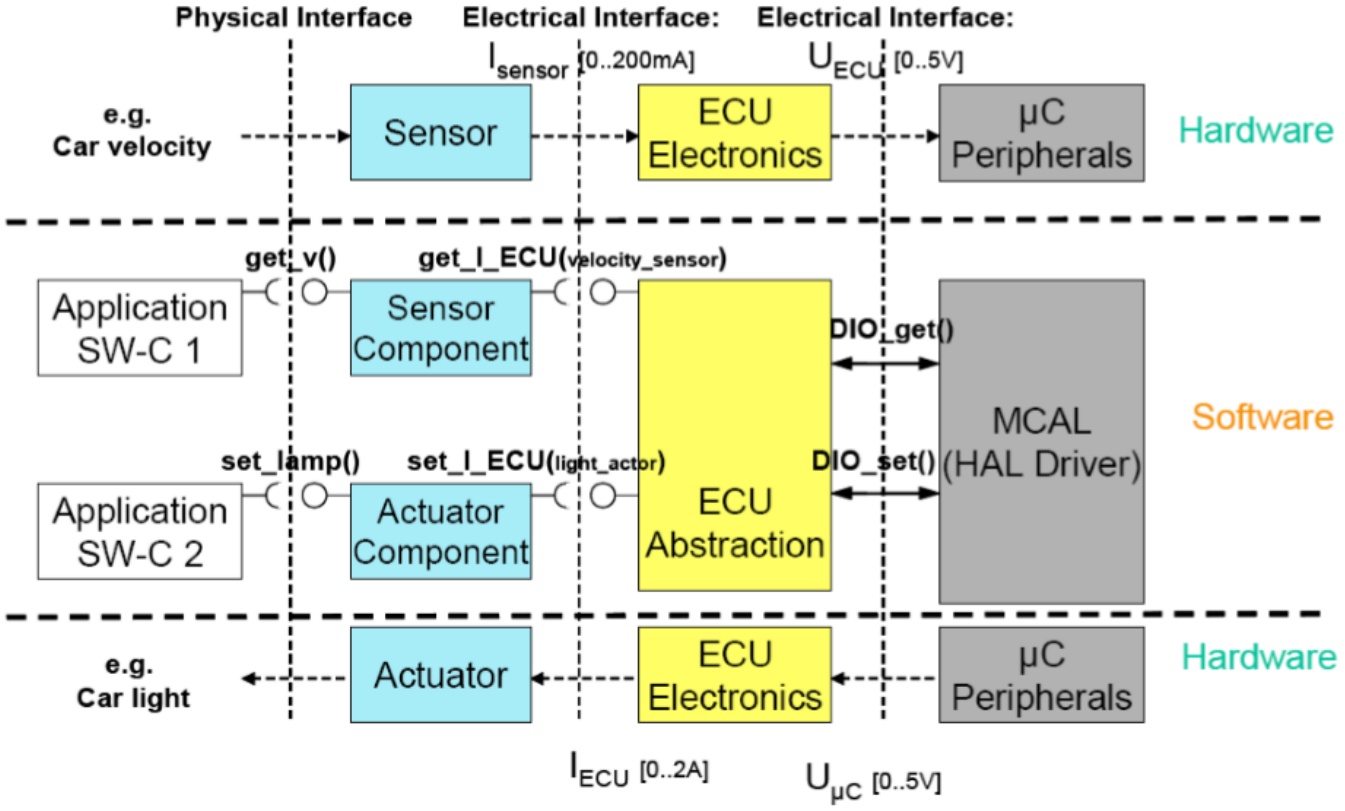
- AUTOSAR software components communicate via the Virtual Functional Bus. They need ways to express requirements and capabilities with respect to exchanging data, which is currently possible through two kinds of attributes:
  - **Communication attributes**, allow specifying parameters of the communication that **affect the generation of the RTE** or the actual communication taking place at runtime. An example for such an attribute is the aforementioned transfer time over a connector.
  - **Application level attributes**, allow describing properties of exchanged data that **do *not* affect the RTE generation**, but are indicate to the developer how data needs to be processed. An example for this kind of attribute is a flag, whether data is “filtered” or “raw”.

Sensor/Actuator Components

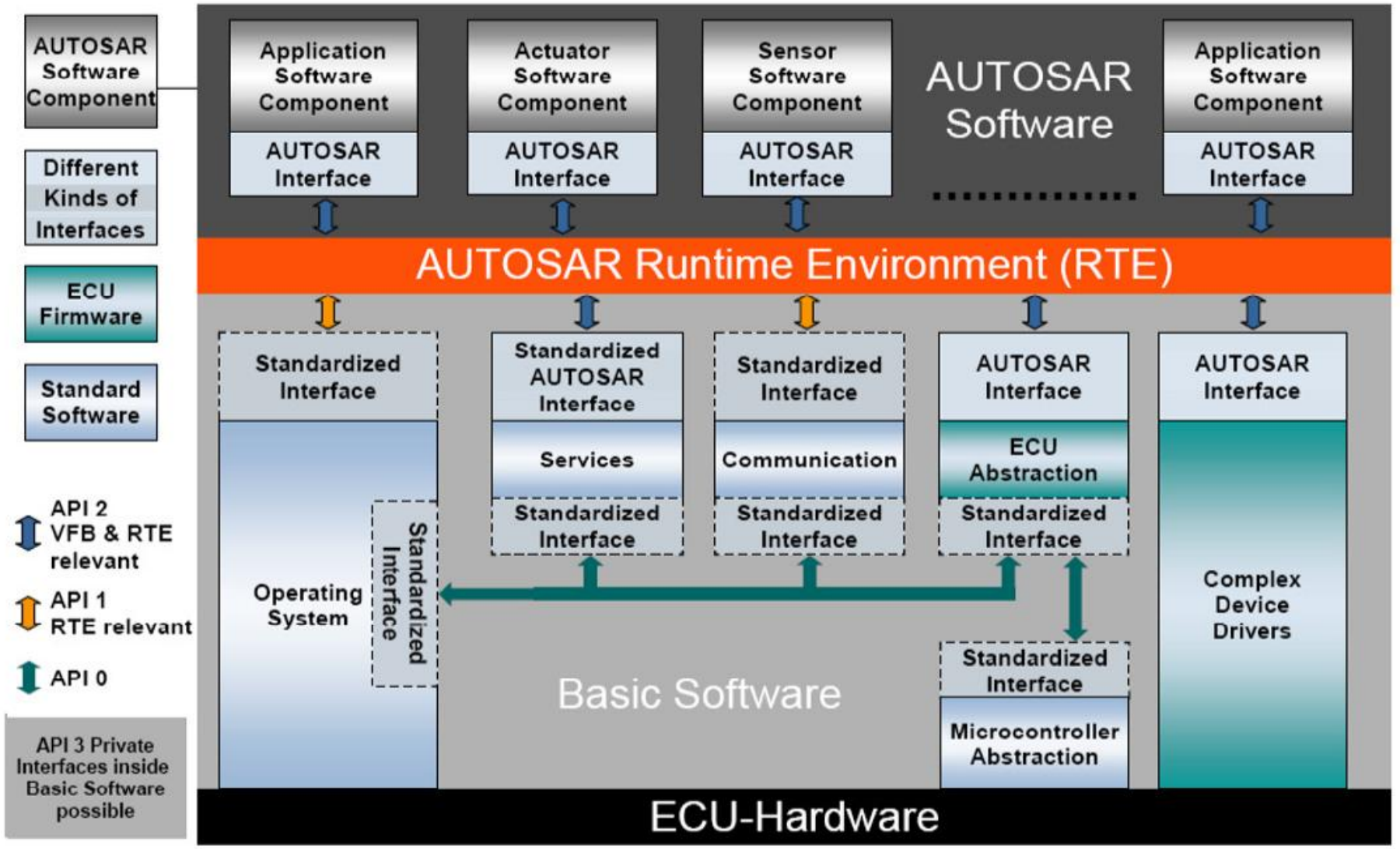
- Sensor/Actuator Components are special AUTOSAR Software Components which encapsulate the dependencies of the application on specific sensors or actuators.
- The AUTOSAR infrastructure takes care of hiding the specifics of the microcontroller (this is done in the **MCAL**, the microcontroller abstraction layer, which is part of the AUTOSAR infrastructure running on the ECU) and the ECU electronics (this is handled by the **ECU-Abstraction** which is also part of the AUTOSAR Basic Software).
- The AUTOSAR infrastructure does **NOT** hide the specifics of sensors and actuators.
- The dependencies on a specific sensor and/or actuator are dealt with in **"Sensor/Actuator Software Component"**, which is a special kind of Software Component. Such a component is independent of the ECU on which it is mapped but is dependent on a specific sensor and/or actuator for which it is designed.
    - **For example**, a "Sensor Component" typically inputs a software representation of the electrical signal at an input-pin of the ECU (e.g. a current produced by the sensor) and outputs the physical quantity measured by the sensor (e.g. the car speed).
- Typically, because of performance issues, such components will need to run on the ECU to which the sensor/actuator is physically connected.

Sensor/Actuator Components

Typical conversion process from physical signals to software signals (e.g. car velocity) and back (e.g. car light).

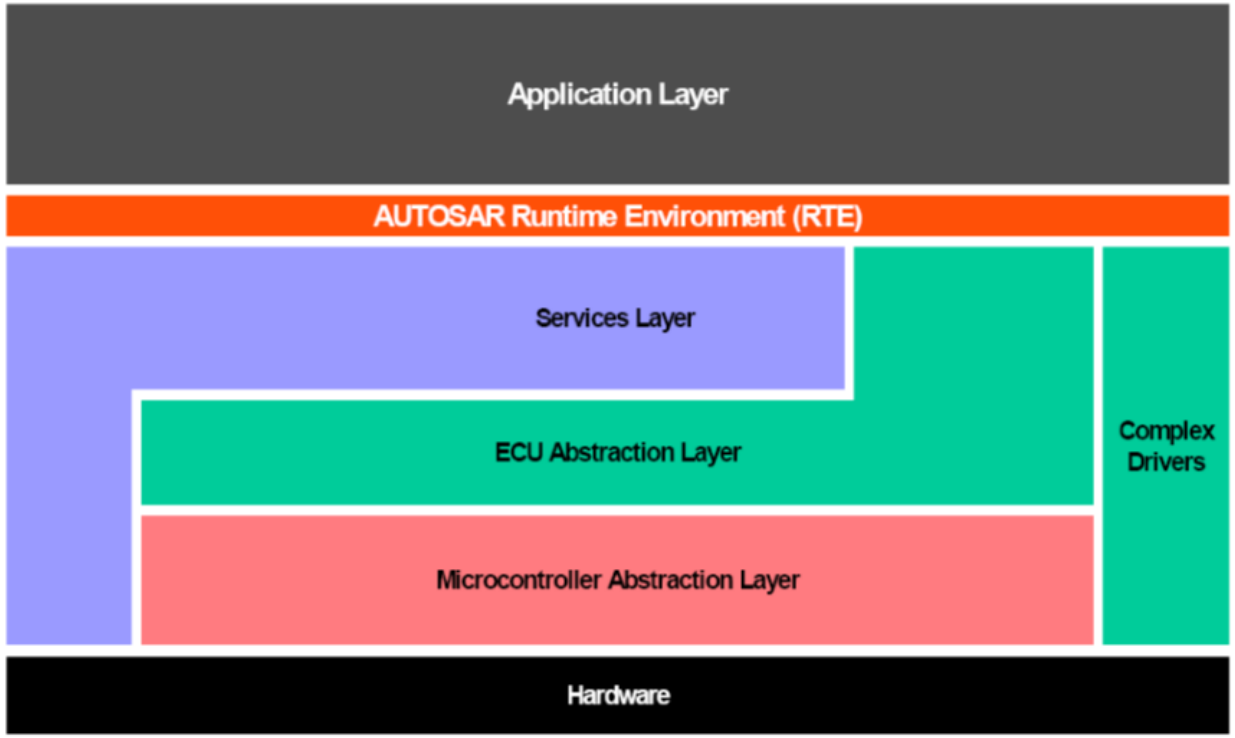


ECU SW Architecture



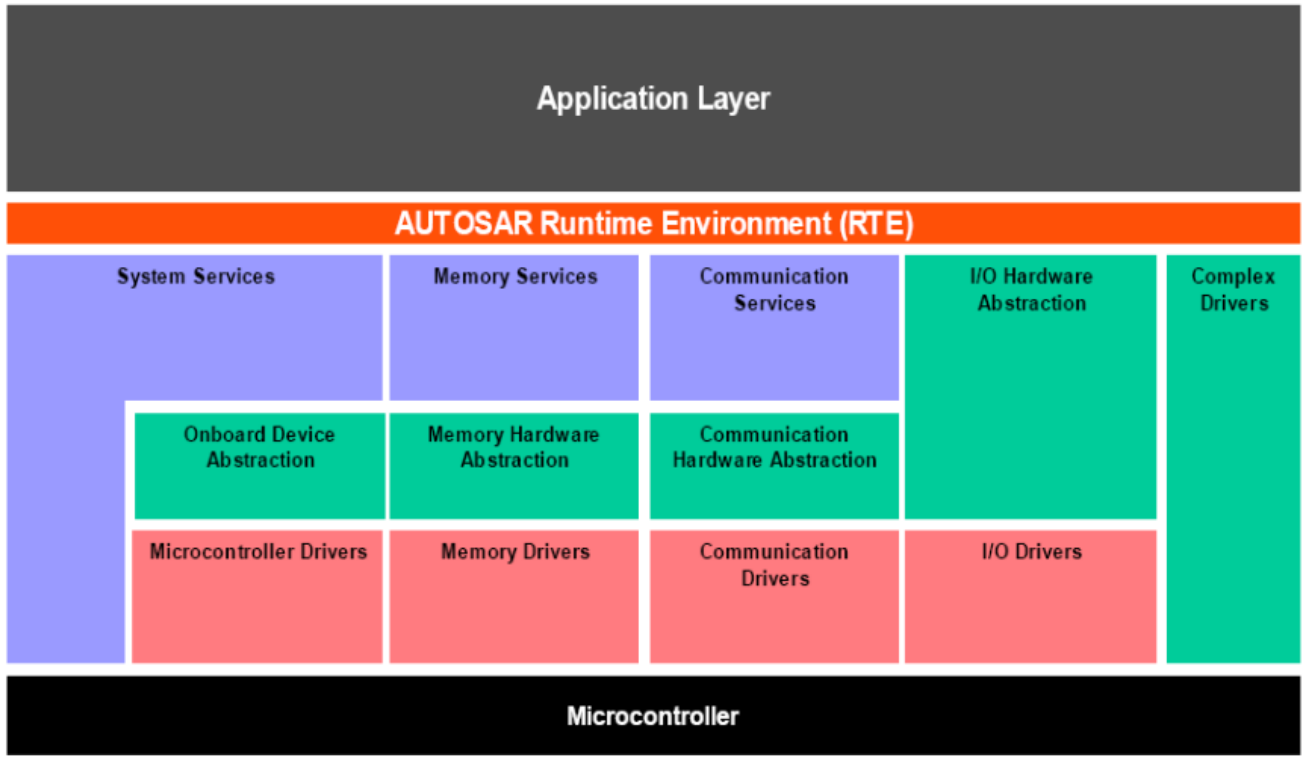
Layered SW Architecture

- A layered architecture has been developed within AUTOSAR to enable a clear and structured interface definition and a well defined abstraction of the hardware.
- The architecture is structured in **5 layers plus the Complex Drivers**.



Basic SW Architecture

- **Refined Layered Architecture (Basic Software)**
- The layered architecture has been further refined in the area of Basic Software. Around 80 Basic Software modules have been defined (11 main blocks plus Complex Drivers).

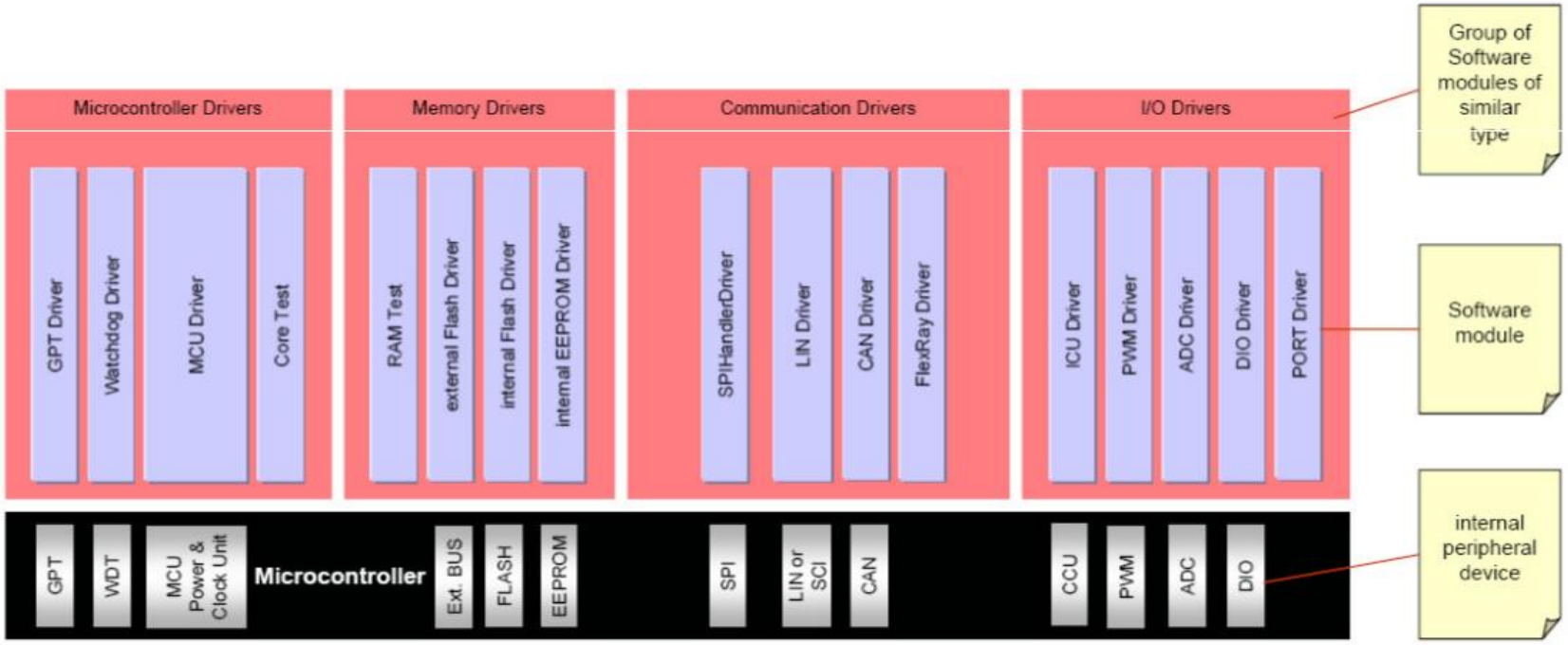


Microcontroller Abstraction Layer(MCAL)

- Access to the microcontroller registers is routed through the **Microcontroller Abstraction Layer (MCAL)**.
- MCAL is a hardware specific layer that ensures a standard interface to the Basic Software. It manages the microcontroller peripherals and provides the components of the Basic Software with microcontroller independent values. MCAL implements notification mechanisms to support the distribution of commands, responses and information to processes.
- **It can include:**
  - Digital I/O (DIO)
  - Analog/Digital Converter (ADC)
  - Pulse Width (De)Modulator (PWM, PWD)
  - EEPROM (EEP)
  - Flash (FLS)
  - Capture Compare Unit (CCU)
  - Watchdog Timer (WDT)
  - Serial Peripheral Interface (SPI)
  - I2C Bus (IIC)

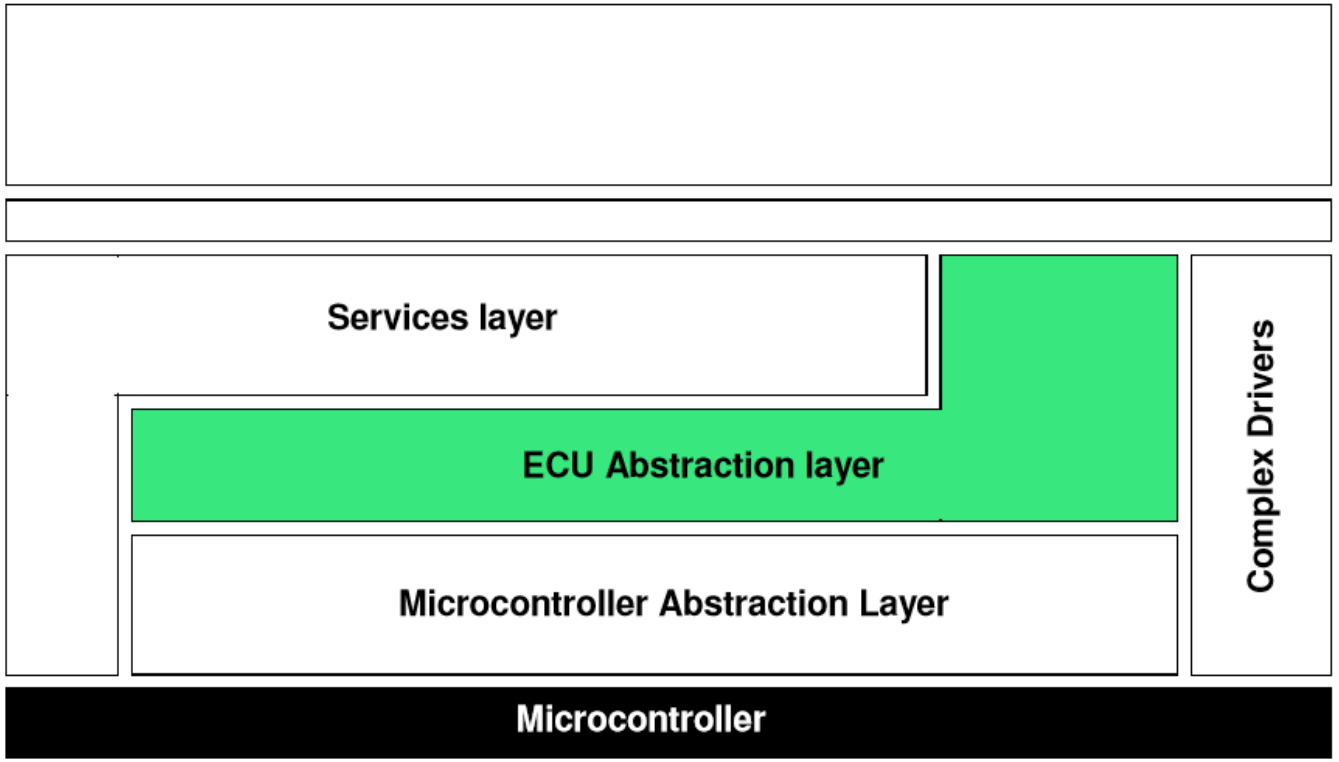
Microcontroller Abstraction Layer(MCAL)

- The Microcontroller Abstraction Layer is the lowest layer of the Basic Software. It contains drivers, with direct access to the micro-Computer internal peripherals and memory mapped micro-Computer external devices.

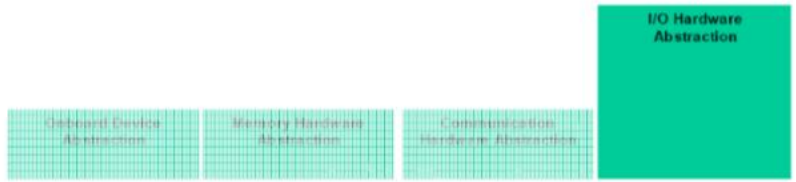


ECU Abstraction Layer

- The **ECU Abstraction Layer** provides a software interface to the electrical values of any specific ECU in order to decouple higher-level software from all underlying hardware dependencies.



ECU Abstraction Layer



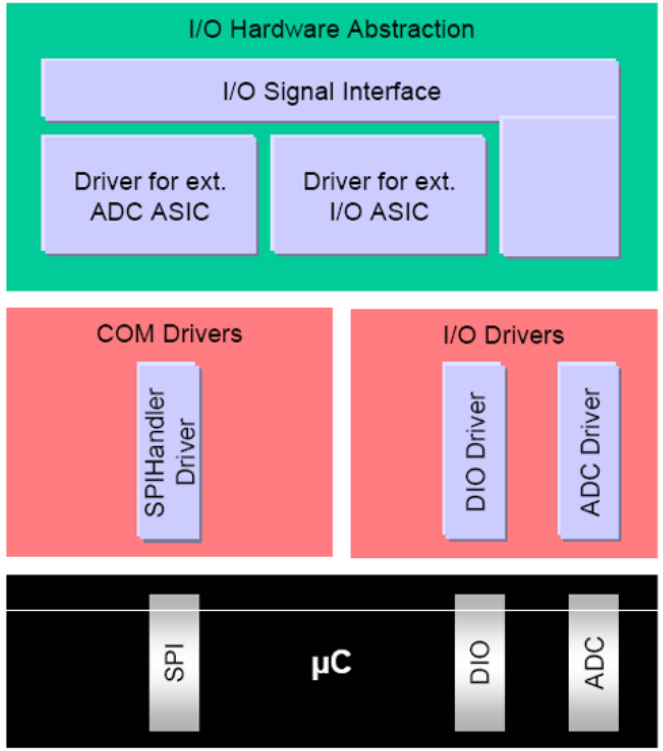
I/O Hardware Abstraction:

A group of modules which abstracts from the location of peripheral I/O devices (on-chip or on-board) and the ECU hardware layout (e.g. µC pin connections and signal level inversions).

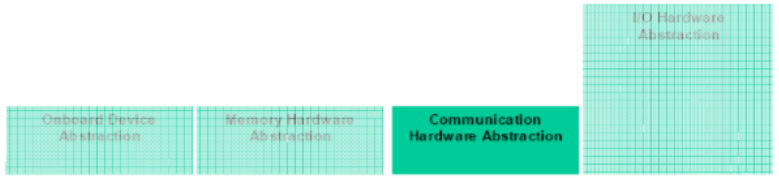
The I/O hardware abstraction does not abstract from the sensors/actuators! I/O devices are accessed via an I/O signal interface.

The task of this group of modules is

- to represent I/O signals as they are connected to the ECU hardware (e.g. current, voltage, frequency), and
- to hide ECU hardware and layout properties from higher software layers.



ECU Abstraction Layer



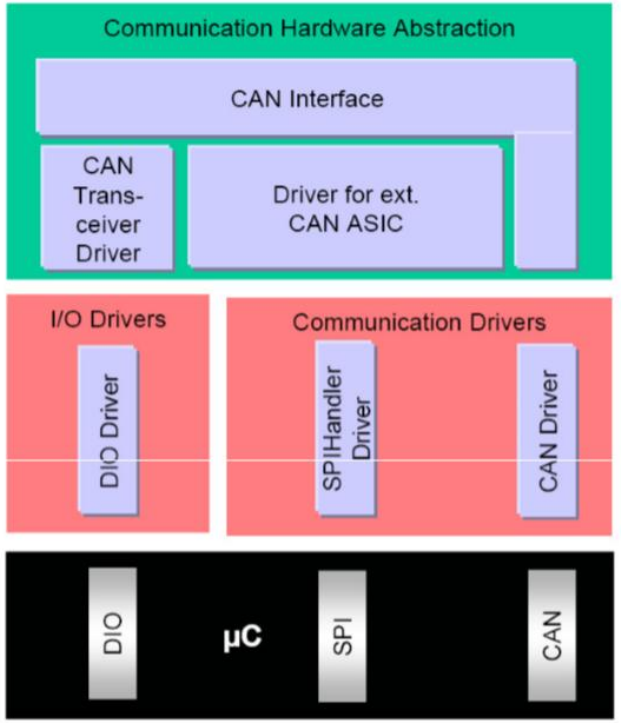
Communication HW abstraction

A group of modules which abstracts from the location of communication controllers and the ECU hardware layout. For all communication systems a specific communication hardware abstraction is required (e.g. for LIN, CAN, MOST, FlexRay).

- Example: An ECU has a microcontroller with 2 internal CAN channels and an additional on-board ASIC with 4 CAN controllers. The CAN-ASIC is connected to the microcontroller via SPI.

The communication drivers are accessed via bus specific interfaces (e.g. CAN Interface). That means the access to the CAN controller should be regardless of whether it is located inside the microcontroller, externally to it, or whether it is connected via SPI.

The task of this group of modules is to provide equal mechanisms to access a bus channel regardless of its location (on-chip / on-board).



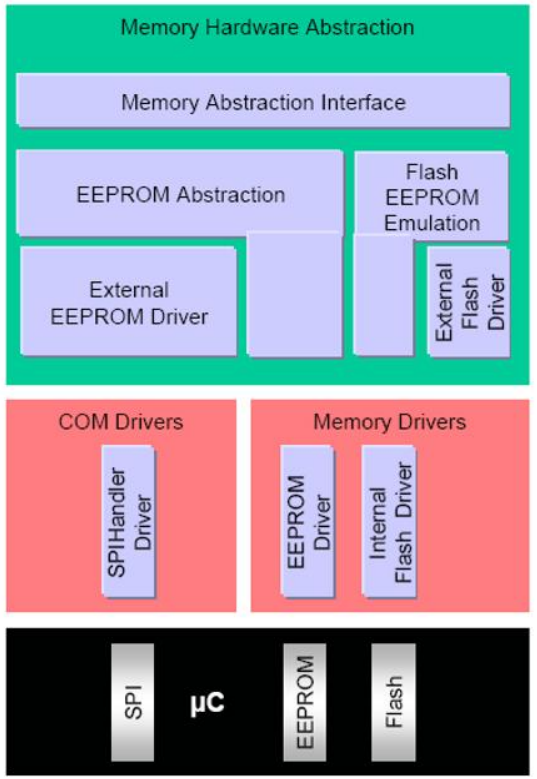
ECU Abstraction Layer

Memory HW Abstraction

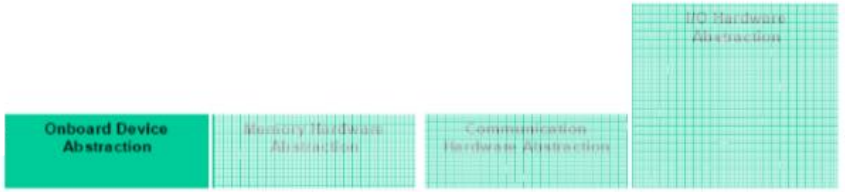
A group of modules which abstracts from the location of peripheral memory devices (on-chip or on-board) and the ECU hardware layout.

- Example: on-chip EEPROM and external EEPROM devices should be accessible via an equal mechanism.

The memory drivers are accessed via memory specific interfaces (e.g. EEPROM Interface). The task of this group of modules is to provide equal mechanisms to access internal (on-chip) and external (onboard) memory devices.



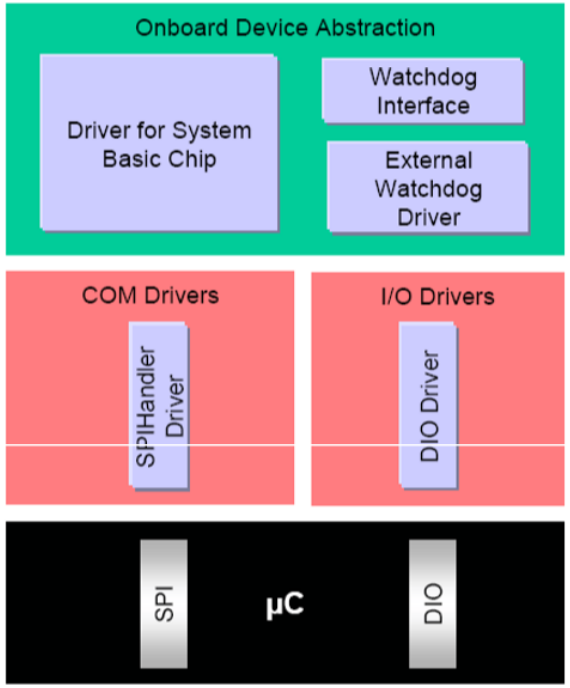
ECU Abstraction Layer



Onboard Device Abstraction

Contains drivers for ECU onboard devices which cannot be seen as sensors or actuators like system basic chip, external watchdog etc. Those drivers access the ECU onboard devices via the µC abstraction layer.

The task of this group of modules is to abstract from ECU specific onboard devices.



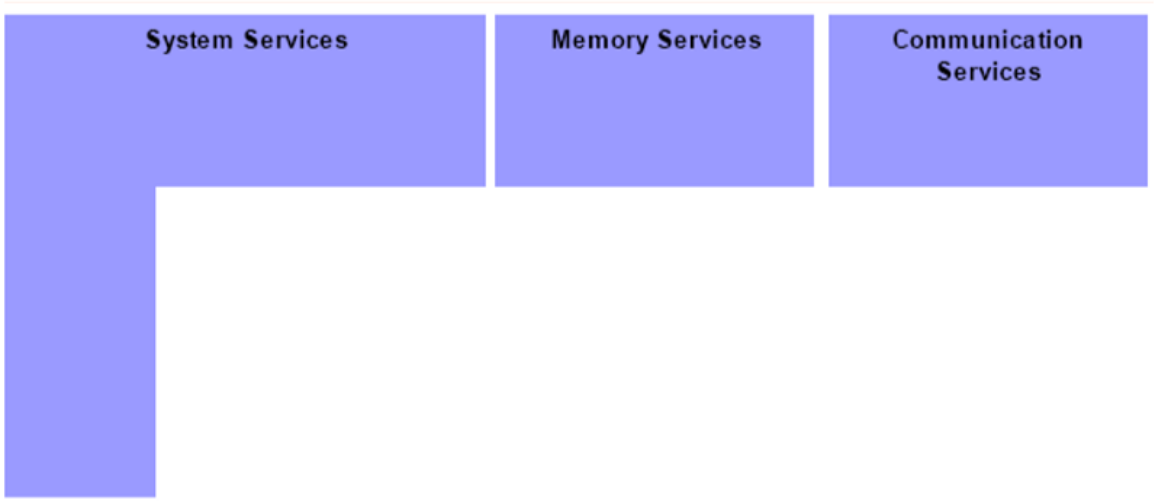
Service Layer

The service layer consists out of 3 different parts:

Communication Services

Memory Services

System Services

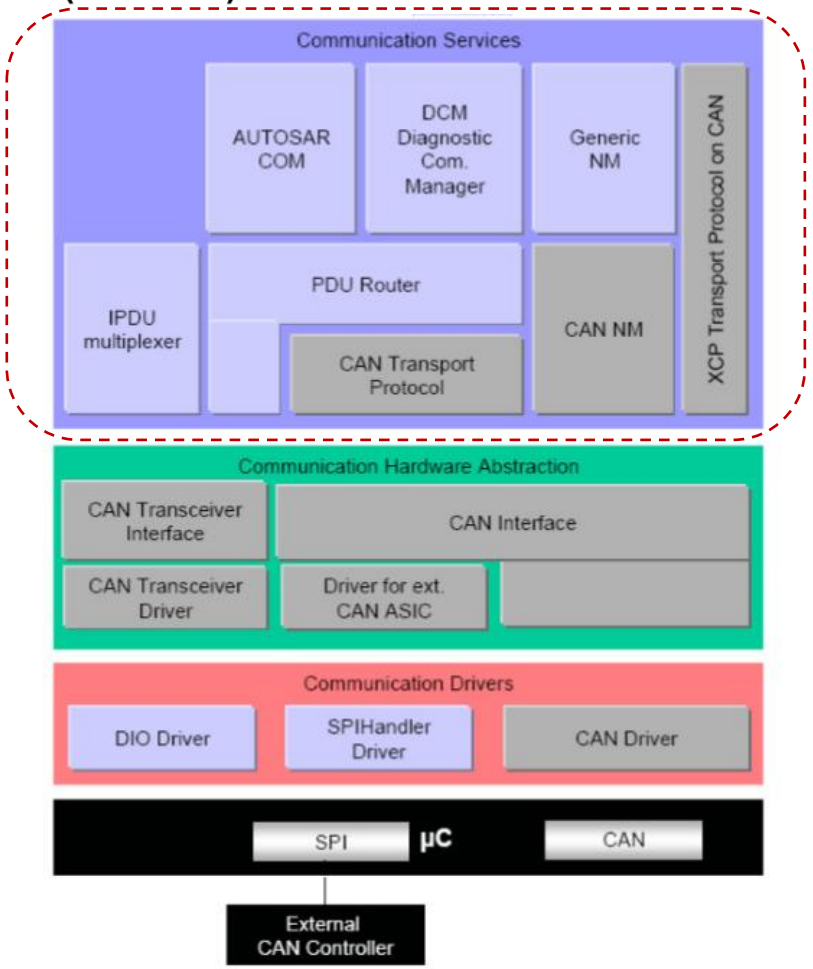


Service Layer – Communication

- The communication services are a group of modules for vehicle network communication (**CAN, LIN, FlexRay and MOST**). They are interfacing with the communication drivers via the communication hardware abstraction.

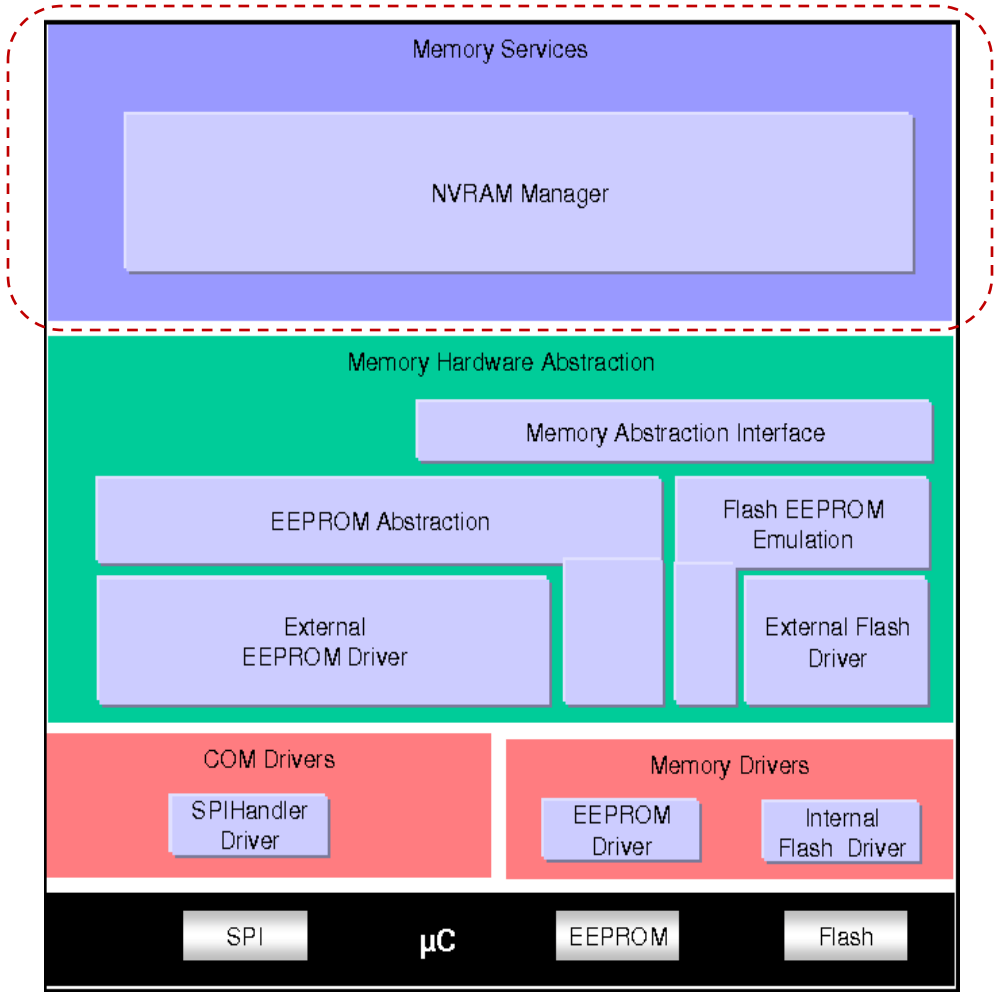
- **The task of this group of modules is:**
  - to provide a **uniform interface** to the vehicle network for communication between different applications,
  - to provide **uniform services** for network management,
  - to provide a **uniform interface** to the vehicle network for **diagnostic communication**, and
  - to **hide protocol and message properties** from the application.

An example (CAN bus):



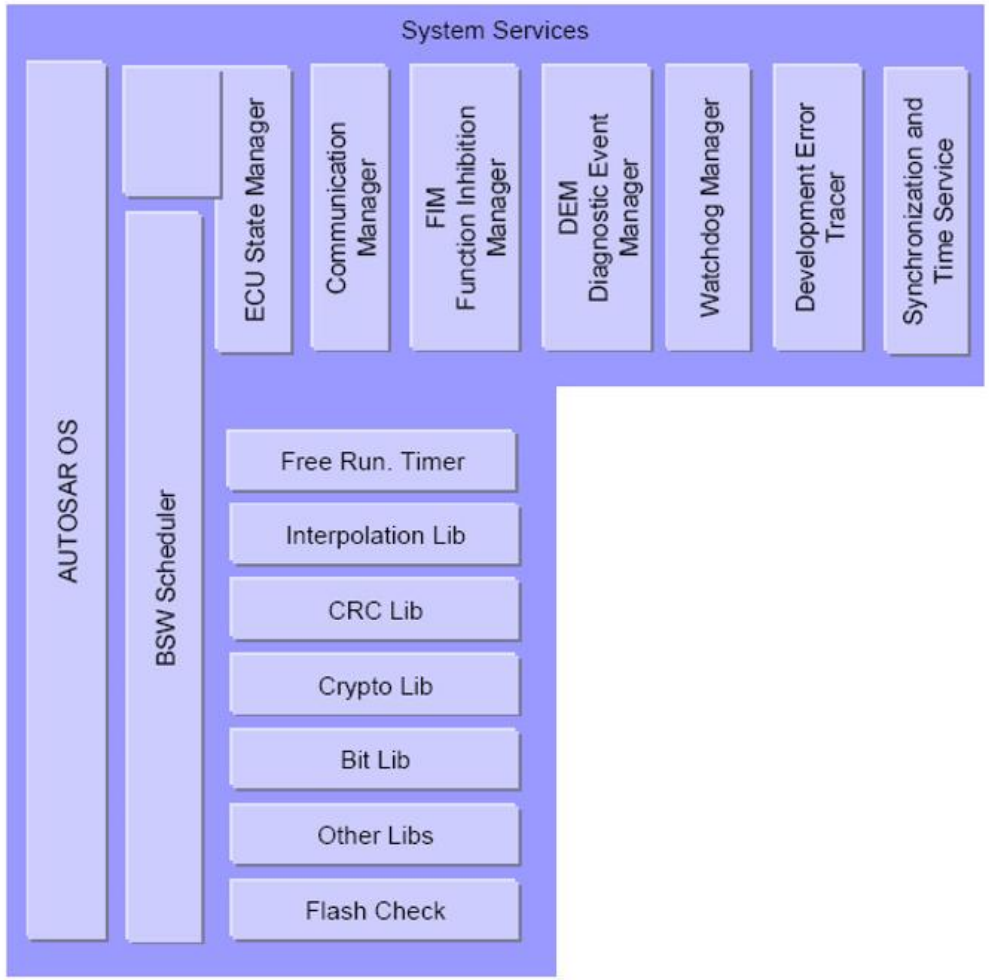
Service Layer – Memory

- A group of modules responsible for the management of non-volatile data (read/write from different memory drivers). The **NVRAM manager** provides a RAM mirror as data interface to the application for fast read access.
- **The task of this group of modules is:**
  - to provide non-volatile data to the application in a **uniform way**,
  - to **abstract from memory locations and properties**, and
  - to provide mechanisms for non-volatile data management like **saving, loading, checksum protection and verification, reliable storage** etc.



Service Layer – System

- The system services are a group of modules and functions which can be used by modules of all layers.
- Examples are real-time operating system, error manager and library functions (like CRC, interpolation etc.).
- Some of these services are:
  - Micro-Computer dependent (like OS),
  - ECU hardware and/or application dependent (like ECU state manager, DCM),
  - or hardware and Micro-Computer independent.
- The task of this group of modules is to provide basic services for application and Basic Software modules.

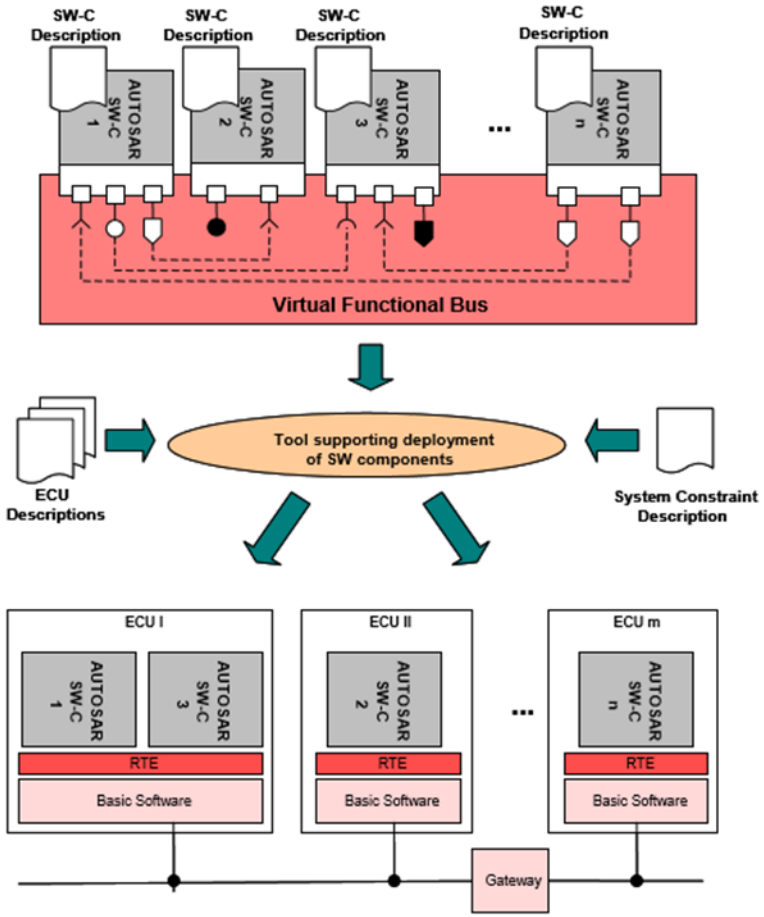


Complex Driver

- The Complex Device Driver is a loosely coupled container, where specific software implementations can be placed. The only requirement to the software parts is that the interface to the AUTOSAR world has to be implemented according to the AUTOSAR port and interface specifications.
- Complex Sensor and Actuator Control
- The main task of the complex drivers is to implement complex sensor evaluation and actuator control with direct access to the micro-Computer using specific interrupts and/or complex micro-Computer peripherals (like PCP, TPU), e.g.
  - injection control
  - electric valve control
  - incremental position detection

Overall Methodology

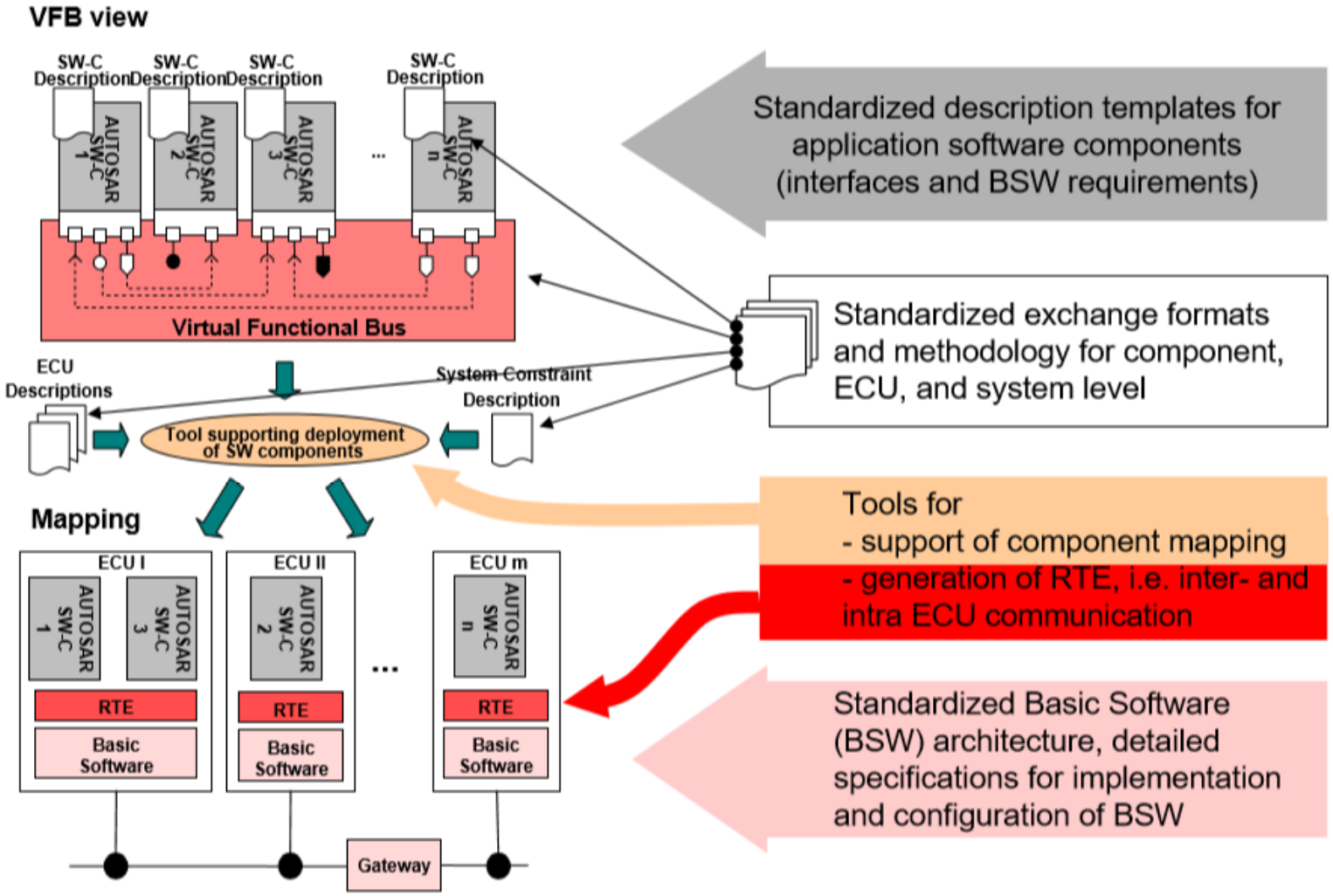
Following the AUTOSAR Methodology, the E/E architecture is derived from the formal description of software and hardware components.



- Functional software is described formally in terms of “Software Components” (SW-C).
- Using “Software Component Descriptions“ as input, the „Virtual Functional Bus“ validates the interaction of all components and interfaces before software implementation.
- Mapping of “Software Components” to ECUs and configuration of basic software.
- The AUTOSAR Methodology supports the generation of an E/E architecture.

Overall Methodology

Derive E/E architecture from formal descriptions of soft- and hardware components

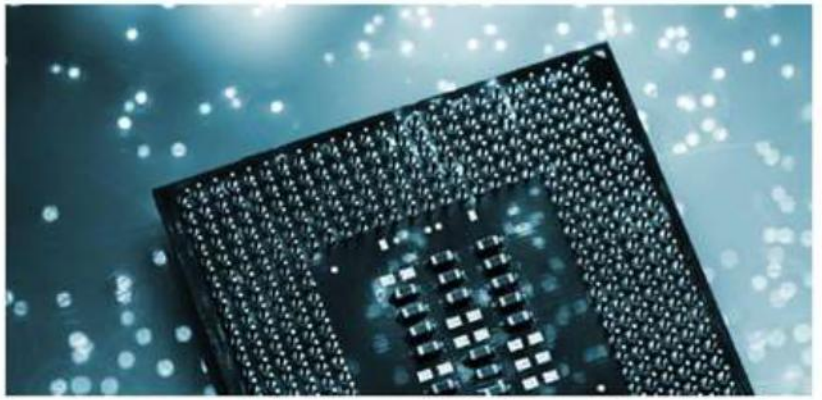


Game changer for Autosar

- Highly automated driving

- Car-2-X applications
- Internet of Things and cloud services

- Increasing data rates



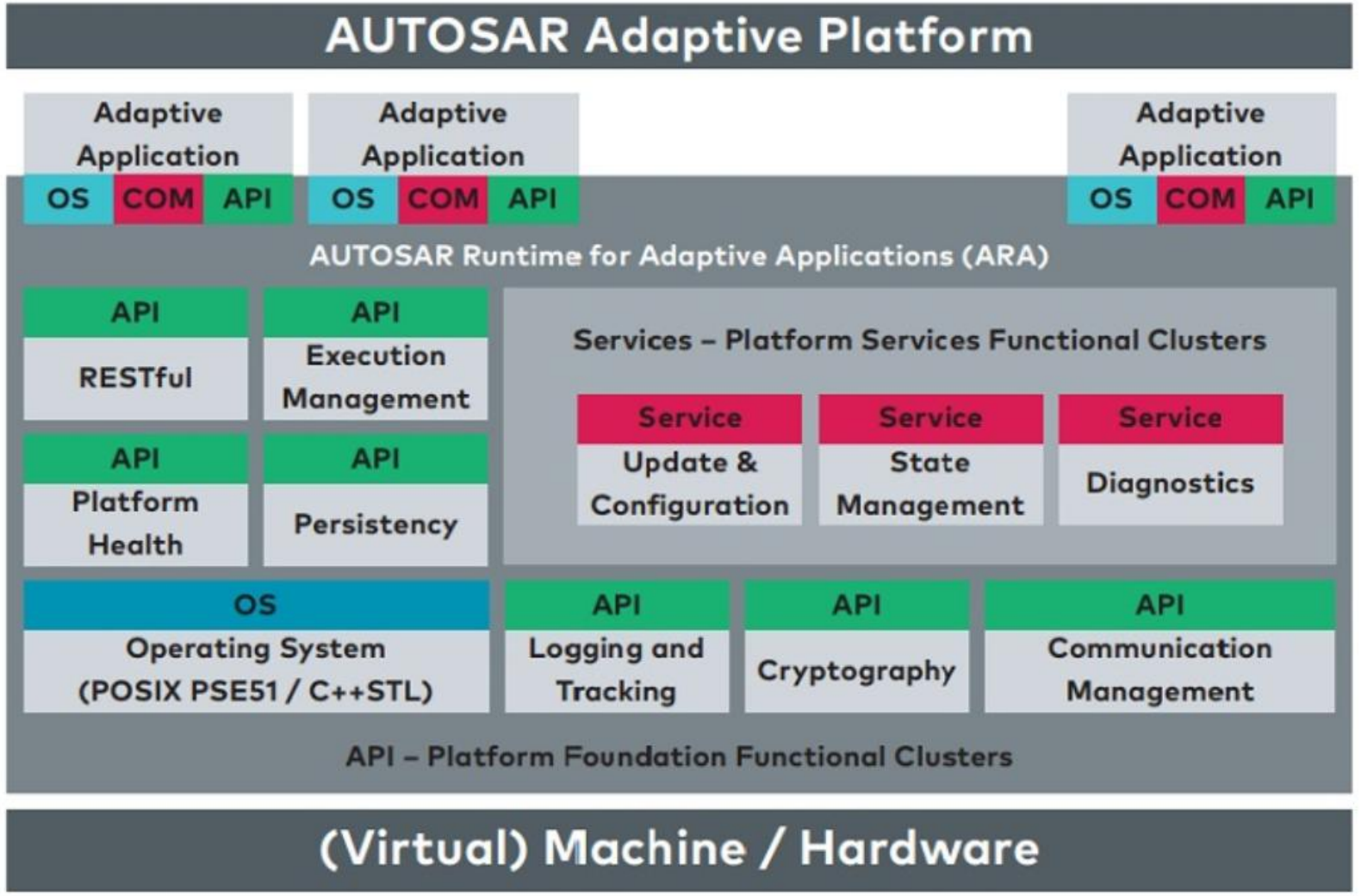
- New processor technologies

AVLAB AUTONOMOUS VEHICLE

Autosar Adaptive

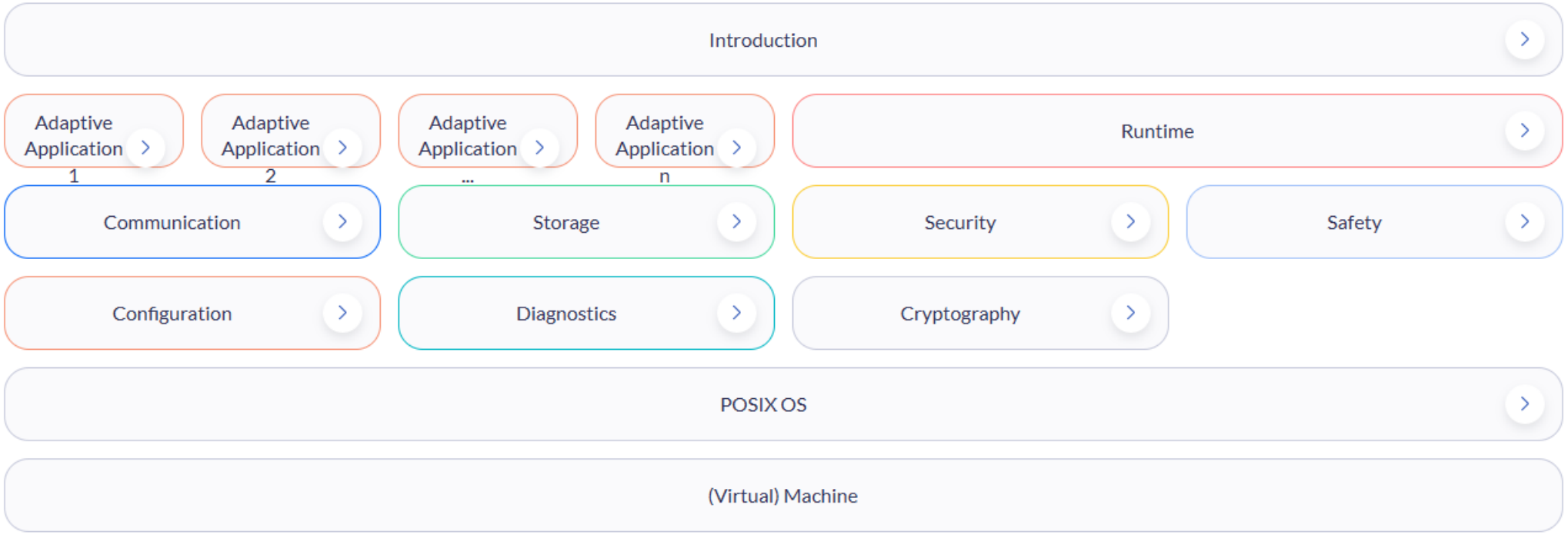
- The AUTOSAR Adaptive Platform implements the AUTOSAR Runtime for Adaptive Applications (ARA). Two types of interfaces are available, services and APIs. The platform consists of functional clusters which are grouped in Services and the AUTOSAR Adaptive Basis.
- Functional clusters...
  - Assemble functionalities of the Adaptive Platform
  - Define clustering of requirements specification
  - Describe behavior of software platform from application and network perspective
  - But do not constrain the final SW design of the architecture implementing the Adaptive Platform.
- Functional clusters in AUTOSAR Adaptive Platform Basis have to have at least one instance per (virtual) machine while services may be distributed in the in-car network
- In comparison to the AUTOSAR Classic Platform the AUTOSAR Runtime Environment for the Adaptive Platform dynamically links services and clients during runtime.

Autosar Adaptive



Autosar Adaptive

- https://www.autosar.org/standards/adaptive-platform/
- Current Release : AUTOSAR adaptive release R25-11



Autosar Classic vs. Adaptive

구분	Classic Platform (CP)	Adaptive Platform (AP)
기반 OS	OSEK 기반	POSIX 51 기반
실행 방식	ROM 기반 실행, 단일 주소 공간, Early binding	RAM 기반 실행, 가상 주소 공간 (MMU 지원)
통신 방식	신호 기반 (Signal based) 통신	서비스 지향 (Service Oriented) 통신
주요 타겟	Deeply embedded (임베디드 제어)	ADAS 및 자율주행 (AD)
주요 용도	Sensors and Actuators Control	High Performance Computing
안전 등급	최대 ASIL-D 지원	최대 ASIL-B 지원
지원 언어	C 언어	C++ 언어
최초 릴리스	2004년	2017년 3월
표준 범위	Specification Only	Specification and Code base
구현체	독자적인(Proprietary) AUTOSAR 구현	Vector, EB 등에서 독자 제품 준비

Thank you for Attention

Q & A

AVLAB
AUTONOMOUS VEHICLE